RPX

RPA/RPX Kernel

精确重构证明规范手册

Reformulation Proof Assembly
Reformulation Proof eXecutable

数学模型 · Contract · 推导与联合扩展
文本汇编 · 二进制格式 · 完整证书
Loader · 执行语义 · 异常与符合性

当前实现参考 · MCM 1 · RPA/RPX 1–10

2026 年 10 月 13 日

依据当前安全 Rust kernel 的实现重写。
V1–V8 已冻结；V9/V10 已实现，协议仍未冻结。

阅读约定与文档状态

RPA 是 *Reformulation Proof Assembly*；RPX 是 *Reformulation Proof eXecutable*。`.rpa` 是带版本头的严格 JSON 证书，`.rpx` 是同一证明对象的顺序二进制编码。MCM 保存完整的精确有理系数模型。调用者另行提供源模型、目标模型和所要求的 contract；证书不能自行降低这个要求。本手册说明当前安全 Rust kernel 如何核验普通有理系数 MILP 的重构关系，包括完整可行向量、目标关系、投影、双射、精确点见证及严格改进。

实现基线与协议状态

本版于 2026 年 10 月 13 日根据当前仓库实现重写，取代 2026 年 10 月 9 日的 D0 设计手册。实现基线为 `crates/core/src/`：数学对象见 `model.rs` 与 `proof.rs`，执行器见 `verify.rs`，文件编解码见 `codec.rs` 与 `rpa.rs`，公开接口见 `api.rs`，命令行见 `main.rs`。静态联合扩展分别由 flow、interval、menu、signed、subflow 模块实现；point、injectivity、level 模块提供对应的精确 contract 检查。

对象	当前状态
MCM	只接受版本 1；与 RPX 的增量版本独立。
RPA / RPX 1–8	已实现、已冻结；后续版本沿用既有字段与数学意义。
RPA / RPX 9	已实现的目标水平域工作草案，尚未冻结。
RPA / RPX 10	已实现的严格改进、完整像排除、联合 subflow 与显式半空间域工作草案，尚未冻结。
D0 / v0.1	历史设计来源，既不是可接受的文本版本，也不是当前文件 ABI。

实现完成与协议冻结是两个不同决定。本手册完整描述已实现的 V9/V10 字段和规则，不会因草案身份省略其前置条件，也不把它们宣布为已冻结的发布协议。原 D0 的 section directory、表达式池、隐式 SSA 编号、逐项 mapping/check、yield/merge 和可执行 theorem plugin 均不属于当前接口。附录A保留其设计决定与当前实现的对应关系。

规范性用语与证据

“必须”“不得”“拒绝”描述本版 kernel 的格式约束、数学接受条件和公开行为；“建议”描述证书作者或集成方的工程选择；“可以”描述规则允许的情况。数学语义、字段顺序、引用可见性和完整义务是规范性内容。数据结构示意、性能解释、有限测试、历史测量和审阅记录是说明性证据。一个文件通过汇编只说明能够编码，不等于证明通过验证。

`docs/CURRENT_FORMAT.md` 汇总冻结版本的布局；`docs/FORMAT_V9_DRAFT.md` 与 `docs/FORMAT_V10_DRAFT.md` 保留工作草案的边界。各联合扩展的独立数学论证及审阅来源在相关章节和附录列出。`CURRENT_FORMAT.md` 的原字节由交付 manifest 固定，保留其交付时点的旧手册说明；本版手册状态以本页为准，冻结布局及历史验收记录不因重写手册而改变。若发现手册与源代码不同，应将其作为需要修复的文档或实现问题，不能让证书通过含糊解释、未定义行为或自动放宽检查取得接受结果。

可信边界

验证依赖实际完整模型、精确算术、静态推导规则和受审阅的联合存在引理。可信实现包含 MPS/LP 导入器、MCM/RPX decoder、有理数库、表达式代数、执行器及构造扩展模型的 builder。builder 的旧 scope 前提检查和模型合法性检查并不能独立证明其生成的所有新坐标同时可行；该同时存在性质来自各自的可信引理。

RPA 作者、Python 生成器、求解器输出、审计 JSON、文件名和摘要都不能授予数学事实。哈希用于记录输入来源；kernel 的接受条件仍然检查实际传入模型上的每一个前提和义务。库的纯验证路径不读取文件系统、环境或时钟；文件操作与计时由 API 外层承担。测试与独立有限 oracle 是实现证据，不能替代对 Rust 实现及依赖的形式化证明。

阅读顺序与符合性范围

数学审阅从第一章和第二章开始：先确定所要求的命题，再检查每个完整输出向量及分支、扩展、反证的语义。第三章逐项列出推导与终结点的前置条件及拒绝条件；第四章给出全部二进制记录，第五章说明真实输入方言、资源上限、API 和诊断，第六章定义文本语法。第七章提供可重复运行的完整实例，第八章区分逻辑工作量、生成成本和进程测量。

Verifier 接受所支持版本时，检查完整文件、独立 request、全部前提与实际模型义务；不得把跳过的分支、失败的规则或资源耗尽报告为 `VERIFIED`。

Assembler / disassembler 负责严格文本与二进制的转换；不承担独立数学认证，重新编码可以选择最低所需版本。

证书作者 安排已完成的推导与完整 mapping，保证每个可达叶使用同一个联合输出向量；不能借用另一方向、兄弟分支或另一文件的局部事实。

集成方 明确传入完整模型、contract 和资源策略，依据 status 判断结果，依据具体 contract 解释其数学结论。

本 kernel 不提供 proof search、一般 MILP 求解器、任意跳转、动态 theorem registry、通用共享表达式 DAG、专用 batch 编码或数值解重建命令。当前语料的完整验收不意味着任意重构都有紧凑证书；`INVALID` 或 `RESOURCE_LIMIT` 也不意味着相应重构关系在数学上不成立。

目录

阅读约定与文档状态 **i**

实现基线与协议状态 . . . i

规范性用语与证据 . . . i

可信边界 . . . ii

阅读顺序与符合性范围 . . . ii

第一章 数学模型与合约 **1**

1.1 完整模型与精确数域 . . . 1

1.2 规范仿射表达式与事实 . . . 2

1.2.1 三种事实 . . . 2

1.2.2 模型事实 . . . 2

1.3 方向 claim 与调用者绑定 . . . 2

1.3.1 总构造与目标关系 . . . 3

1.3.2 坐标与仿射观察 . . . 3

1.3.3 仿射 inverse . . . 3

1.3.4 目标层与仿射半空间：未冻结 V9/V10 . . . 3

1.4 点合约：存在性与分离 . . . 4

1.5 完整域双射合约 . . . 4

1.6 严格改进合约：未冻结 V10 . . . 5

1.7 共同扩展的逻辑边界 . . . 5

第二章 执行语义与验收 **6**

2.1 证明对象与上下文 . . . 6

2.2 引用解析与局部定义 . . . 6

2.2.1 父子作用域 . . . 7

2.3 目标命题检查与 Exact . . . 7

2.4 步骤序列与穷尽分类 . . . 7

2.4.1 Cases 的共同事实 . . . 8

2.4.2 Split 的完整子证明 . . . 8

2.5 完整 Return 与义务生成 . . . 8

2.5.1 改进 Return 的差异 . . . 9

2.6 lift 的执行协议 . . . 9

2.7 矛盾端点与上下文许可 . . . 9

2.8 请求级执行顺序与拒绝 . . . 10

2.9 正确性不变式 10

第三章 操作与推理规则参考 11

3.1 操作总表与共同条件 11
3.2 Lin：精确线性组合 11
3.3 Round：整数表达式精确舍入 12
3.4 Drop：局部可见性删除 12
3.5 EqPair：精确反对称性 12
3.6 Cases 与 DerivationEnd：局部穷尽证明 12
3.7 Plotkin：带已验近对 clique 的码上界 13
3.8 Return、Split 与 Absurd 13
3.8.1 Return：完整输出 13
3.8.2 Split：穷尽完整子 Block 13
3.8.3 Absurd：恒定矛盾 14
3.9 共同扩展的公共约定 14
3.10 FlowLift：共同源商品分解 14
3.11 IntervalLift：独立实区间选择 15
3.12 MenuLift：可选二元菜单 15
3.13 SignedLift：有激活的有界实数正负分解 15
3.14 SubflowLift：有需求部分的共同有界子流 16
3.15 ImproveReturn 与 PositiveRefutation 16
3.15.1 ImproveReturn：同模型严格下降 16
3.15.2 PositiveRefutation：与严格根条件矛盾 17

第四章 MCM 与 RPX 二进制格式 18

4.1 通用记法与文件头 18
4.2 规范有理数与表达式 19
4.2.1 Q 的布局与拒绝规则 19
4.2.2 Expr 的布局 19
4.3 MCM 1 模型布局 19
4.4 RPX 外层与 Claim 20
4.5 Ref、Kind 与 Step 21
4.6 Block 与 Derivation 21
4.7 联合扩展 witness 的逐字段布局 22
4.8 Point、Bijection 与 Improvement 23
4.9 版本选择、降版与资源预检 24

第五章 Loader、接口与可信边界 26

5.1 入口、处理顺序与判定 26

5.2 可信精确模型导入 . . . 27
5.2.1 共同规范化与模型身份 . . . 27
5.2.2 MPS 的 sections、cards 与排列 . . . 27
5.2.3 MPS bounds、markers 与 ranges . . . 27
5.3 LP v1 的自足文本规则 . . . 28
5.3.1 sections、名称与 coordinate order . . . 28
5.3.2 表达式、续行与关系 . . . 29
5.3.3 bounds 与 integrality . . . 29
5.4 精确数值与各层资源边界 . . . 30
5.5 真实 CLI 与 Rust façade . . . 30
5.6 报告、诊断与模型 JSON . . . 31
5.7 信任结论与外部证据 . . . 32

第六章 RPA 文本汇编与工具链 33
6.1 精确 header 与 JSON 词法 . . . 33
6.2 精确 scalar 字符串 . . . 33
6.3 顶层 Certificate 与独立 Request . . . 34
6.3.1 Claim 的所有 keys . . . 35
6.4 Expr、Ref 与显式事实 IDs . . . 35
6.5 Step 与局部 Derivation 的 JSON variants . . . 36
6.6 Block End 与 witness records . . . 37
6.7 Point、Bijection 与 Improvement contracts . . . 38
6.8 一套可直接执行的最小程序 . . . 39
6.9 规范反汇编、失败与实现依据 . . . 39

第七章 完整证书示例与验收案例 41
7.1 复核入口与结果 . . . 41
7.2 示例 A：删除冗余上界行 . . . 41
7.2.1 完整模型与调用者要求 . . . 41
7.2.2 完整 RPA 与核验 . . . 42
7.3 示例 B：利用等式消去一个变量 . . . 43
7.3.1 完整模型与调用者要求 . . . 43
7.3.2 完整 RPA 与核验 . . . 44
7.4 示例 C：分段绝对值 mapping . . . 44
7.4.1 完整模型与调用者要求 . . . 44
7.4.2 完整 RPA 与核验 . . . 45
7.5 每份示例的拒绝与资源检查 . . . 46
7.6 完整 corpus：严格改进与条件最优投影 . . . 46
7.7 完整 corpus：联合 subflow 与 point contracts . . . 47

7.8 实际案例的可重复入口 48

第八章 执行成本、资源预算与测量 49

8.1 公开计数的确切含义 49
8.2 共享预算与可复核的资源结果 50
8.3 代换、精确累加与表示成本 50
8.4 Shared scopes 的实现与内存边界 51
8.5 时钟字段与端到端成本 51
8.6 已有实验：明确保留适用范围 52
8.6.1 共享状态与精确算术 52
8.6.2 V10 旧任务 regression 与顺序敏感性 52
8.6.3 新能力绝对 baseline 52
8.7 复现与下一次优化的登记 53

附录 A 设计沿革、冻结范围与后续登记 54

A.1 D0 提案 P-01 至 P-07 的实际处置 54
A.2 实现版本 V1 至 V10 的能力历史 55
A.3 D0 开放项 O-01 至 O-05 的当前状态 56
A.4 V9/V10 冻结前继续保留的边界 57
A.5 手册修订记录与维护准则 58

第一章　数学模型与合约

本章规定当前 kernel 的数学对象与成功验证所证明的命题。RPA（Reformulation Proof Assembly）是证明汇编文本，RPX（Reformulation Proof eXecutable）是相应的二进制证书；两者描述相同的证明对象。模型由 MCM 1 表示，证明格式 V1–V8 已冻结；当前源码还实现了未冻结的 V9 目标层与 V10 条件域、严格改进、完整像排除及子流扩展。这些草案功能的实现状态不等于协议冻结。

1.1　完整模型与精确数域

一个模型为

$$M = (V, R, f), \quad V = (v_0, \ldots, v_{n-1}), \quad R = (r_0, \ldots, r_{r-1}),$$

其中每个变量给出整数声明及可选上下界，每个行给出一个仿射表达式及可选上下界，f 为完整的规范化最小化目标。变量与行分别从零编号；编号指向实际向量，名称只作元数据。验证器接收调用者提供的两个**完整**模型，必须检查全部结构和数值。RPX 没有用模型摘要、选定行集合或模型名称代替这些模型的机制。

全部有限标量属于精确数域 $\mathbb{Q}$，采用

$$q = p/d, \qquad p \in \mathbb{Z}, \quad d \in \mathbb{Z}_{>0}, \quad \gcd(|p|, d) = 1.$$

零的规范值为 0/1。分母为零、非规范分数及超过资源界的标量拒绝。十进制、科学计数法和分数字面量均表示精确有理数；例如 `0.125` $= 1/8$，`1e-6` $= 1/1000000$。模型系数为有理数不意味着连续坐标也限于有理数：证明必须对实数域成立。**必须：**决定接受的加法、乘法、取整、代换及比较遵循精确算术，不使用可行性容差。

变量 v_i 表示 $x_i \in \mathbb{R}$，带条件

$$l_i \le x_i \le u_i,$$

其中缺失下界或上界分别表示负无穷或正无穷。仅实际存在的界参与事实与义务。变量 domain 只有连续与整数两种；所谓二元变量是声明整数且具有相应有效界的变量，没有第三种 MCM domain 标签。行 j 表示

$$L_j \le a_j(x) \le U_j,$$

a_j 可以带常数，可以为空支持；缺失的一侧不产生条件。两侧有限且相等时是等式行。行常数不必移入界。**必须：**矛盾的变量界或行界仍是合法模型数据，表示可能为空的可行域；验证器不得修复或删除它们。

目标为

$$f_M(x) = c_M + \sum_i d_{Mi} x_i.$$

导入最大化模型时，整个目标连同常数取负。目标常数进入目标比较、目标层和点检查。MPS 与 LP 导入属于可信转换边界；不支持的原始文件构造应在导入时拒绝，不能在证明中赋予其未定义的含义。

模型名称、变量名称和行名称必须非空、为 ASCII、无控制字节且不超过 1024 字节。变量名称在变量向量内唯一，行名称在行向量内唯一；两个命名空间独立。名称的唯一性是结构条件，名称本身不授予任何数学事实。

1.2 规范仿射表达式与事实

表达式为

$$e(x) = c + \sum_{k=0}^{t-1} a_k x_{i_k}, \quad 0 \le i_0 < \cdots < i_{t-1} < n, \quad a_k \neq 0.$$

常数与系数为规范有理数。空支持表示常数表达式。序列必须已经严格有序、无重复、无零系数；二进制解码与证明验证不会隐式修复输入表达式。内部算术可合并同坐标并消去零项，所得结果仍须规范且满足资源界。表达式直接作为记录出现，没有 ExprId 池、隐式变量名或映射寄存器。原始 claim 的表达式只能使用原始模型坐标；局部证明表达式可使用当前扩展模型坐标。

两个表达式相等是常数与全部坐标系数的精确相等。逻辑上等价的 $e \le 0$ 与 $2e \le 0$ 不自动成为同一 proposition。哈希只可帮助查找，不能代替精确表达式比较。

1.2.1 三种事实

类型	数学意义
`Le`	$e(x) \le 0$，记为 LE(e)。
`Eq`	$e(x) = 0$，记为 EQ(e)。
`Int`	$e(x) \in \mathbb{Z}$，记为 INT(e)。

事实在产生时具有作用域：当前模型、当前路径假设以及活跃局部定义共同决定其有效域。一个经过检查的 EQ(e) 可以在目标命题检查处证明同表达式的 LE(e) 或 INT(e)，因为零是不正整数。这不改变各条指令的原始前提种类要求；例如 Round 仍须一个 Le 和一个 Int。证明语言不含一般严格不等式事实；V10 的严格域由独立改进合约保存。

1.2.2 模型事实

当前输入模型的静态引用按下式生成：

$$\begin{aligned}
&\texttt{VarLower}(i): && \mathrm{LE}(l_i - x_i),\\
&\texttt{VarUpper}(i): && \mathrm{LE}(x_i - u_i),\\
&\texttt{VarInt}(i): && \mathrm{INT}(x_i),\\
&\texttt{RowLower}(j): && \mathrm{LE}(L_j - a_j(x)),\\
&\texttt{RowUpper}(j): && \mathrm{LE}(a_j(x) - U_j),\\
&\texttt{RowEq}(j): && \mathrm{EQ}(a_j(x) - L_j).
\end{aligned}$$

界引用要求相应有限界存在；RowEq 要求两侧有限且相等。VarInt 要求变量声明整数并且当前 input integer mode 开启。引用无效编号、缺失界或不满足这些条件时拒绝。当前模型包括已合法加入的条件行和 lift 子体中的扩展行；新行的权威来自相应可信规则，不能在规则自己的前提证明中使用。

1.3 方向 claim 与调用者绑定

令 S、T 为完整 source、target 模型。每个 Claim 各自选择方向、输入/输出整数模式和附加关系：`reverse=false` 表示 $S \to T$，为 true 表示 $T \to S$。记该方向的实际输入输出模型为 I, O。整数模式开启时保留模型全部整数声明；LP 模式只去掉整数条件，保留全部界和行。定义相应可行域

$$\mathcal{F}_b(M) = \{x \in \mathbb{R}^{n_M} : \text{全部界和行成立，若} b \text{ 则全部声明整数坐标属于} \mathbb{Z}\}.$$

调用者独立提供 Request；其中有序 claim、点合约、双射合约和改进合约，必须逐项、逐字段等于证书中相应声明，不能由证书削弱。每个方向恰有一个同序根 Block。空 Request 拒绝；只有双射数组而没有其方向也不能形成有效请求。旧式 Claim 数组请求不包含点、双射或改进。

1.3.1 总构造与目标关系

一个方向证明

$$\forall x \in D_I \quad \exists y \in \mathcal{F}_{b_O}(O): \quad \text{请求的全部附加关系成立},$$

D_I 默认是 $\mathcal{F}_{b_I}(I)$，条件域见下节。Return 给出一个完整仿射输出向量；Split 可以让不同分支使用不同向量，lift 可以先共同选取辅助坐标，再由子体构造输出。任何情况下，每个叶子的全部输出条件必须由**同一个**输出向量满足。观察集合为空也不允许省略输出变量、输出行或整数条件。

目标关系为以下三种：

`None`	不请求目标关系。
`Equal`	$f_O(y) - f_I(x) = 0$。
`NonIncreasing`	$f_O(y) - f_I(x) \le 0$。

`input_objective_lower`=L 另请求 $L - f_I(x) \le 0$。它在默认域上是全局下界，在受限域上仅是条件下界。方向不可行时命题可以为空真；这些字段本身不保证非空或最优值达到。

定义 $\mathrm{val}_b(M) = \inf_{x \in \mathcal{F}_b(M)} f_M(x)$, $\inf \varnothing = +\infty$, 允许 $-\infty$ 和不达到的下确界。无条件 NonIncreasing 方向给出 $\mathrm{val}_{b_O}(O) \le \mathrm{val}_{b_I}(I)$；两个模式兼容的相反方向若均不增，则给出扩展最优值相等。两个相反的总可行构造给出可行性等价。以上推论不适用于随意把受限域称为整个模型。

1.3.2 坐标与仿射观察

`projection` 为有序 (i, o) 对，要求 $y_o = x_i$。每个编号在对应原始模型中有效，同一对不得重复；不同对可共享输入或输出坐标。`affine_projection` 为有序 (p, q) 对，要求 $q(y) = p(x)$。其两侧分别在原始输入/输出维度验证。这些关系表达调用者选择的观察值保留；不要求观察覆盖整个模型，也不自行声明集合相等。

1.3.3 仿射 inverse

`inverse`=j 选择另一条相反且输入输出模式互换兼容的 claim。当前和第 j 条根端点均须为普通 Return，完整根映射分别记为 F, G。当前方向要额外证明每个原始输入坐标的

$$G_i(F(x)) - x_i = 0.$$

它不自动证明另一侧组合 $F(G(y)) = y$；该方向须另有请求。如果第 j 条输入域限定目标层 L，当前 Return 还必须证明 $f_O(F(x)) = L$，以保证组合进入 G 的已证域。V10 暂不支持当前或被引用 claim 带 halfspace 的 legacy inverse；遇到即拒绝。

1.3.4 目标层与仿射半空间：未冻结 V9/V10

`input_objective_level`=L 将该方向域改为 $f_I(x) = L$ 的截面；`input_halfspace`=H 再要求 $H(x) \le 0$。两者可同时存在：

$$D_I = \mathcal{F}_{b_I}(I) \cap \{f_I = L, \text{若请求}\} \cap \{H \le 0, \text{若请求}\}.$$

H 只能使用原始输入坐标。这些限制由调用者声明并逐字段绑定，不证明 H 在全域有效、L 为最优值或该截面非空。构造保留完整原始模型，在原始 r 行后先加入目标等式（若有），再加入半空间上界行（若有）。因此目标等式为 RowEq(r)，半空间为 RowUpper(r) 或 RowUpper($r+1$)；半空间没有下界，不能直接用 RowEq 把 $H \le 0$ 变成 $H = 0$。每条 claim 从自己的原始模型重新建立限制，不向其他方向或改进块传播。

1.4 点合约：存在性与分离

PointWitness 提供所选完整模型的全部有理坐标；每个数值和表达式均精确检查。点是补充合约，不替代方向证明。所有方向先验完毕，再处理点。

Feasible。 选择 source 或 target、LP 或整数模式，以及可选目标值 v。验收证明该完整点属于所选实际模型，若请求 v 则其完整目标恰为 v。它不继承任何方向的条件域。全局下界与同值可行点可一起证明最优值达到，但仅有 Feasible 不证明最优性。

Strictness。 引用一条已证明的无条件 LP-to-LP 包含方向。两侧维度必须相同，其坐标 projection 恰为完整坐标置换。提供的点 y 必须输出 LP 可行；按该置换反构的 x 必须违反输入的实际界或行。这在置换识别的坐标空间中证明真包含；整数性违反不能作 LP 分离。附加 refutation 拒绝。

FiberStrictness。 引用一条无条件 LP-to-LP 方向，可用部分坐标和仿射观察。完整输出可行点 y 固定全部观察，输入 fiber 为

$$\{x \in \mathcal{F}_0(I) : x_i = y_o \ ((i,o) \in \text{projection}), \quad p(x) = q(y) \ ((p,q) \in \text{affine_projection})\}.$$

证书必须在这个完整 fiber 模型上提供独立的 LP 矛盾证明。原始输入行后先加坐标等式，再加仿射等式，顺序均与请求一致；不得只检查选定输入坐标或猜选一个补全。反证从新局部状态开始，只允许 Split/Absurd 端点；整数声明不激活，lift 不可用。因此证明观察像的真包含。观察为空时仍须反证整个输入 LP，不能把空观察当成免费分离。Strictness 与 FiberStrictness 均拒绝目标层或半空间受限的包含 claim。

ImageExclusion：未冻结 V10。 调用者提供完整仿射 map 和点所在一侧。map 恰有对侧每个坐标的表达式，且只用点所在原始模型坐标。验收证明一个完整 LP 可行点的这个指定完整像不属于对侧 LP。先完整验证 map，再精确计算所有像坐标，最终要求实际界或行违反。两侧均松弛整数性，分数坐标本身不足以排除。它不请求包含索引，不证明全局包含、真包含或部分映射的所有补全都不可行。Feasible 与 ImageExclusion 也不允许附加 refutation。

1.5 完整域双射合约

V7 的 BijectionClaim 绑定两个有序方向索引 (a, b)，称为 forward 和 reverse。这里名称只表示角色，forward 的实际 `reverse` 位仍可为 true。索引须有效、彼此不同，同一有序对不得重复；方向必须相反且模式互换兼容。两条 claim 均须无目标层和半空间限制。forward 根端点须为普通 Return，其实际完整仿射映射记为 $F : A \to B$。reverse 可分支或扩展，但每个 Return 必须证明全部

$$F_j(G(y)) - y_j = 0 \qquad (j = 0, \ldots, n_B - 1).$$

这给出 F 的满射性。它们与 reverse 本身完整输出义务同时检查，不能以菜单局部唯一性或少量观察代替。

另有独立 EqualityProof 证明 F 的单射性。kernel 建立两份完整 A，坐标依次为 x, x'，保留各自全部界、行和原整数声明，以原 forward input mode 检查两份 domain，再加每个 $F_j(x) - F_j(x') = 0$。合成模型目标为零；同一映射的常数在差中抵消。证明从新状态开始，必须依原坐标顺序证明全部 $x_i - x'_i = 0$。无观察到的连续辅助坐标也不得省略。两个总方向、上述满射组合与单射证明一起建立该实际完整域映射的双射，空域也遵循同一定义。这与单条 legacy inverse 的单侧组合是不同合约。

1.6 严格改进合约：未冻结 V10

ImprovementClaim 选择一个完整实际模型 M、整数模式 b 及原始坐标表达式 P。验收命题为

$$\forall x \in \mathcal{F}_b(M),\ P(x) > 0 \quad \exists y \in \mathcal{F}_b(M) : f_M(y) < f_M(x).$$

没有隐含目标层限制。各 ImprovementWitness 与 Request 同序同值，使用新局部状态；根假设 0 为弱事实 LE$(-P)$，严格部分由不可变根合约另行保存。

ImproveReturn 必须给出同一个实际模型的完整输出向量，并证明其全部可行性义务及

$$\mathrm{LE}\big(f_M(y) - f_M(x) + h(x)\big).$$

`Constant`(q) 给出 $h = q$，`Predicate`(q) 给出 $h = qP$；两者都要求精确 $q > 0$。后者适用于 $P > 0$ 没有统一正下界的情形。严格域保证 $h > 0$，从而得到严格下降。普通 Return 在改进块中拒绝；PositiveRefutation 只在改进块中可用，其精确义务为 LE(P)，与根的 $P > 0$ 矛盾。它不证明弱闭包 $P \geq 0$ 不可行，也不能在局部 Cases 中代替普通事实证明。

该命题允许空域、无界目标和不达到的下确界。它推出每个已达到的最小点满足 $P \leq 0$，不推出最小点存在或最优值。条件方向 $P \leq 0$ 与严格改进可以共同覆盖所有已达到最小点；欲得 source/target 的最优关系仍需另行完整证明目标和恢复方向，不能把某一请求独自改名为复合结论。

1.7 共同扩展的逻辑边界

各 lift 的核心是可信存在命题

$$M(x) \wedge \text{路径假设} \wedge \text{已验前提} \implies \exists z\ E(x, z).$$

它保留旧模型与目标，仅向当前子体加入新坐标、界和行。每项前提都在同一个扩展前作用域证明，所有项目的选择同时存在。子体必须再对一个共同输出向量证明整个实际输出模型与合约；不能用每条行各自不同的存在选择拼出可行性。具体数学前提、生成顺序与构造论证见第三章。

这些规则是可信推理代码，不是动态定理注册、求解器回调或外部插件。成功检查前提、验证生成模型结构及运行子体，均不独立证明扩展引理或 Rust builder 正确。数学论证、独立构造 oracle 和攻击测试是不同证据，不能宣称它们构成实现的形式化证明。

第二章　执行语义与验收

kernel 执行的是一棵有限证明树。节点先执行有序 Steps，再执行恰好一个 End。每个 Return 一次提交完整输出向量及完整检查引用序列；没有逐条写入映射、单独 CHECK 指令、义务完成位图或分支后的映射汇合。局部 Cases 与终结 Split 的区别是前者输出一条共同事实，后者输出两棵完整方向证明。

2.1　证明对象与上下文

对象	内容及用途
Block	有序 Steps 和一个 End；证明完整构造、共同扩展或矛盾。
Derivation	有序 Steps 和 Witness/Absurd；只证明 Cases 指定的一条 proposition。
EqualityProof	有序 Steps 和有序 EQ 检查引用；用于双射的完整域单射性。

普通方向的抽象上下文记为

$$\mathcal{C} = (I, O, C, \Gamma, \Delta, \mathcal{B}),$$

I 是当前输入模型，O 是实际完整输出模型，C 是不可削弱的调用者 claim，Γ 是有序路径假设，Δ 是活跃局部事实，$\mathcal{B}$ 是共享资源预算。另有请求绑定的 inverse、双射组合以及改进根条件等不可变信息。进入目标层/半空间域时 I 加入精确限制行；进入 lift 子体时 I 保留其全部前缀并扩展。O、原始请求表达式和附加关系的含义不随扩展改变。

每个根方向从空 Γ, Δ 开始。单射性 EqualityProof、fiber 反证和各改进块也各用新状态，不能借用方向中得出的局部事实。改进块唯一预置假设为 $\Gamma_0 = \mathrm{LE}(-P)$；严格根条件 $P > 0$ 另存。在同一次 Request 内，各阶段共享工作预算，进入新状态并不重置预算。

2.2　引用解析与局部定义

九种 Ref 的完整语义如下。编号均是相应逻辑空间的索引，而非文件偏移。

引用	解析结果与有效条件
`VarLower`(i)	当前 I 的 $\mathrm{LE}(l_i - x_i)$；须存在有限下界。
`VarUpper`(i)	当前 I 的 $\mathrm{LE}(x_i - u_i)$；须存在有限上界。
`VarInt`(i)	当前 I 的 $\mathrm{INT}(x_i)$；须声明整数且 input integer mode 开启。
`RowLower`(j)	$\mathrm{LE}(L_j - a_j)$；须有有限下侧。
`RowUpper`(j)	$\mathrm{LE}(a_j - U_j)$；须有有限上侧。
`RowEq`(j)	$\mathrm{EQ}(a_j - L_j)$；须有相等的两有限侧。
`Local`(id)	Δ 中当前可见的已验定义；已 Drop、未定义或仅在兄弟可见时拒绝。
`Assumption`(k)	Γ_k；0 为最早加入的假设，越界拒绝。
`Exact`	只对 checker 已确定的目标 proposition 做窄规则检查，不解析为普通事实。

Lin、Round、EqPair、Cases、Plotkin 用显式 u32 `id` 发布一个局部事实。ID 不须从零连续编号，也不须递增。发布时该 ID 不得仍然活跃，且活跃数不得超过预算；所有前提、类型和表达式先通过检查。

Drop 按其数组顺序删除当前可见 ID，缺失 ID 或同一数组中重复删除已删 ID 均拒绝。一个 ID 仅在删除后才能复用，新的定义有新的 proposition；引用只读当前定义。

2.2.1 父子作用域

Split 和 Cases 的两个子作用域继承父局部事实与既有假设。子体可删除继承 ID，在该子体内复用 ID，并创建其他事实；这些变化不能改变父或兄弟。两子体各附加本分支假设，退出时撤去该假设。Cases 两子体都成功后，仅在父作用域发布其指定的一条共同结论；子体的其他事实、删除或重定义均不向父发布。Split 是端点，不存在需要合并的父续体。

lift 把当前活跃事实和路径假设带入其唯一子体。旧坐标和旧行的完整前缀不变，因而旧事实仍有效；所有新行只在子体可见。在分支内扩展不会给兄弟增加坐标或行。实现可共享不可变父状态并保存子差量；这种存储选择不能改变上述可见性。

2.3 目标命题检查与 Exact

设 checker 要证明 (K, e)，通常引用 r 解析为 (K_r, e_r)。接受当且仅当 $e_r = e$ 精确成立，且 $K_r = K$ 或 $K_r = \texttt{Eq}$。比例相等、仅在某个点取值相等、ID 相等及显示文本相等均不足。

Exact 不从证书读入命题；checker 先构造 e，再检查：

1. EQ：e 无变量项且常数为零。
2. LE：e 无变量项且常数不正。
3. INT：常数为整数，每项系数为整数，且每个出现坐标声明整数、输入整数模式开启。

常数整数没有坐标前提，在 LP 模式也可接受。非恒真的仿射不等式不能通过 Exact；无模型整数性的非恒定 LP 表达式也不能通过 Exact。生成的菜单或 signed 二元坐标可在相应整数子域满足 INT 的窄规则，生成的 flow、interval 和 subflow 连续坐标不能。

Exact 可出现在 Return/ImproveReturn 检查、单射 EQ 检查、lift 前提和 DerivationEnd Witness，以及改进的 PositiveRefutation 目标检查。它不能作为 Lin（即使乘子为零）、Round、EqPair、Plotkin、整数 Split/Cases 或任何 Absurd 的原始事实操作数。一个已有 Eq 可做目标检查的种类弱化，但不赋予原始操作数自动种类转换。

2.4 步骤序列与穷尽分类

步骤顺序执行；后一步仅能引用当前有效的模型事实、假设与前面活跃定义。各条推理规则见第三章。六种 Step 的命名空间与十种 End、两种 DerivationEnd 的命名空间独立。同一个数字 tag 在不同位置不表示同一操作。

Split 与 Cases 都先验证规范表达式 e。未提交整数引用时，两个分支假设为

$$\mathrm{LE}(e), \qquad \mathrm{LE}(-e).$$

它们覆盖全实数域，$e = 0$ 的边界同时属于两支。提交整数引用时，须解析出种类确为 *Int* 且表达式恰为 e 的事实，然后两假设为

$$\mathrm{LE}(e), \qquad \mathrm{LE}(1 - e).$$

整数域中分别表示 $e \le 0$ 与 $e \ge 1$，穷尽且互斥。不能用 Exact 或 Eq 原始引用冒充该 Int 前提。

2.4.1 Cases 的共同事实

Cases 在父作用域指定 ID、Kind 和 expression；两 Derivation 分别在穷尽分支上证明这同一 proposition，或用普通 Absurd 证明该支为空。Witness 使用目标命题检查规则，因此允许 Exact 以及同表达式的 Eq 种类弱化。两支不能各自选择不同 expression，再要求 checker 猜测共同结论。Derivation 没有 map、Return、lift、PositiveRefutation 或严格改进端点。内部 Steps 仍可递归 Cases；成功后父得到唯一指定事实。

2.4.2 Split 的完整子证明

每个 Split 子 Block 都必须以该上下文允许的端点成功完成。两支 Return 可以给出不同完整输出向量；Absurd 可关闭已证明不可能的支。不能把一个分支只验了变量界、另一分支只验了行的证明合成一个 Return。判为不可能的支仍须先证明合法矛盾；kernel 不按猜测的可达性跳过证明。

2.5 完整 Return 与义务生成

Return 的 `map` 为当前输入坐标上的规范仿射表达式序列 $Y = (Y_0, \ldots, Y_{n_O-1})$，长度必须恰等于实际输出变量数。定义一次完整代换

$$e[Y] = e_0 + \sum_j e_j Y_j.$$

所有义务都使用这个共同 Y。map 的全部表达式先验维度和规范性，checks 再按下列确定顺序逐一消耗。该序列没有由证书提供的对象 ID，也没有可选的输出义务子集。

序	对象	构造的精确命题
1	输出变量，坐标序	每个变量先有限下界 LE($l_j - Y_j$)，再有限上界 LE($Y_j - u_j$)，最后若声明整数且 output integer mode 开启则 INT(Y_j)。
2	输出行，行序	两有限侧相等时只有 EQ($a_j[Y] - L_j$)；否则先有限下侧 LE($L_j - a_j[Y]$)，再有限上侧 LE($a_j[Y] - U_j$)。
3	请求目标	None 不产生；Equal 为 EQ($f_O[Y] - f_I$)；NonIncreasing 为 LE($f_O[Y] - f_I$)。
4	坐标观察，然后仿射观察	各自依请求序：EQ($Y_o - x_i$)，然后 EQ($q[Y] - p$)。
5	legacy inverse	每个原始输入坐标：EQ($G_i[Y] - x_i$)。
6	对侧 inverse 目标层	若被引用 opposing claim 请求层 L，证明 EQ($f_O[Y] - L$)，即使当前 objective 为 None 也不能省略。
7	输入目标下界	若请求 L，证明 LE($L - f_I$)；受限域上的结论是条件下界。
8	V7 双射组合	所有 reverse 索引选中当前方向的 pair，按 Request 双射数组序，每 pair 按 forward 完整输出坐标序证明 EQ($F_j[Y] - x_j$)。

未存在的界不生成义务，但存在的矛盾界不予省略。等式行不是两条 LE，恒等或冗余行也仍有相应检查位置。变量上下界即使相等仍分别产生两条 LE。每项 checks 引用须证明该位置的准确表达式与种类；缺失与多余引用均拒绝。因此引用序列不能重排，也不能靠重复数量来代替表达式核对。

双射组合中的 x_j 是当前方向原始输入坐标，扩展后仍为旧坐标前缀。forward 使用所绑定 claim 的实际根 Return 映射，不能换用证书另行提交的观察 map。legacy inverse 的维度与模式在执行前核对，且要求两侧根 Return。

2.5.1 改进 Return 的差异

改进上下文输出模型就是输入原始完整模型，输入输出整数模式相同。ImproveReturn 先按上表 1、2 验完全部可行性，再只有一项目标义务

$$\mathrm{LE}\big(f_M[Y] - f_M(x) + h(x)\big),$$

$h = q$ 或 qP，精确 $q > 0$。此合约没有 projection、inverse、目标层、输入下界或双射组合字段。当前 Y 可以使用合法 lift 坐标；P 始终保持原始坐标含义。普通 Return 与 ImproveReturn 的上下文不得互换。

2.6 lift 的执行协议

FlowLift、IntervalLift、MenuLift、SignedLift、SubflowLift 均采用同一顺序：

1. 检查允许的上下文、深度、witness 维度、原作用域表达式与标量。
2. 在剩余共享预算内构造显式未验前提和候选扩展模型；完成尺寸、算术和名称检查。
3. 用扩展前的 I, Γ, Δ 逐项验证全部前提，包括冗余或零命题。
4. 只有全部成功，才以候选模型进入唯一 body；保留旧局部事实、假设和请求。

在准备阶段生成的前提记录携带 Ref，不为 Ref 授予权威。任何组不能借用本次同时添加的另一组坐标或新行证明自己的前提。嵌套 lift 的下一层则可合法使用已经建立的上一层坐标。

MenuLift 与 SignedLift 要求 input integer mode；Flow、Interval、Subflow 可用于 LP 或整数模式。普通映射和改进块可用 lift；独立 LP/fiber 反证只允许 Split/Absurd 端点，故拒绝 lift。legacy inverse 的根仿射 Return 要求排除所有 lift 根。V7 双射 reverse 可使用 lift，但仍须每叶完整组合和独立完整域单射证明。

扩展保留原输入名称、全部变量/行前缀与完整目标；新坐标目标系数为零。所有生成名称避开已有名称，编号由追加顺序决定。扩展的存在引理及生成方程属于可信规则本身；上述步骤不运行路径搜索，也没有通用运行时存在性证明器。

2.7 矛盾端点与上下文许可

普通 Absurd 解析一条当前有效原始事实，只接受无变量项的 LE(c) 且 $c > 0$，或 EQ(c) 且 $c \neq 0$。非恒定矛盾猜测、Int 事实和 Exact 都不能关闭一个叶。Absurd 成功表示该当前域为空，不产生局部结论，不修改输出模型。

PositiveRefutation 仅在改进块中检查 LE(P)，与保存的严格根条件矛盾。它没有放宽普通 Absurd，也不赋予局部 Cases 严格事实。边界 $P = 0$ 的 Split 支仍执行；其可由 PositiveRefutation 关闭严格域中的空支。

执行对象	允许的完成形式
普通方向 Block	Return、Split、Absurd、五种 lift。
改进 Block	ImproveReturn、PositiveRefutation、Split、Absurd、五种 lift。
独立 LP/fiber 反证 Block	仅 Split 或 Absurd；Steps 仍可含普通 Cases。
Cases Derivation	Witness 或普通 Absurd。
双射 EqualityProof	全部步骤后，精确数量的有序原坐标 EQ 检查。

2.8 请求级执行顺序与拒绝

完整请求验证先匹配改进、点及双射合约数组，拒绝空请求和不适用的受限 strictness；验证完整 source/target、全部方向声明与对应根 Block。双射索引、方向、模式、重复 pair、无条件域和 forward 根 map 必须先满足结构条件。每个方向独立验证原始坐标观察、inverse 及限制域，随后执行其完整证明树。全部方向成功后，依序执行各双射的单射 EqualityProof，再验全部点，最后验改进块。任一阶段失败，全请求不被验证；早先成功的子阶段不能被当作整体 VERIFIED。

二进制证书在执行前完整解码，含每个 Split/Cases 的两支和所有附加证明。未知 tag、版本不许可的记录、非法保留位、截断和尾随字节不能隐藏在所谓不可达分支。执行也不因该支看似不可行而跳过其步骤；它必须给出允许的终结证明。

拒绝涵盖：错误合约、错误维度/坐标、非法规范数值、无效/失活引用、错误前提种类、非法线性乘子、覆盖活跃 ID、错误整数分类前提、缺失/多余义务、精确表达式不匹配、错误端点上下文以及任何未完成子证明。资源拒绝另外包括工作、活跃事实、嵌套深度、扩展尺寸、项数、分配和有理数位数超限。资源拒绝不能解释为数学命题为假，也不能升级为验证成功。

默认工作预算为 10^9，默认活跃事实 10^6，组合证明深度 64，根深度为 1。Split、Cases 和 lift body 均增加一层；它们共享同一个深度边界。精确中间结果也受标量位数限制，即使后续可消去超大项仍不能绕过。工作计量是确定性的资源策略，不能当作物理内存、CPU 指令数或位复杂度保证。更完整的 codec、文本和模型限制见第五章。

CLI 的结果为 VERIFIED、INVALID、RESOURCE_LIMIT、IO_ERROR，分别退出 0、2、3、4。ASSEMBLED 等辅助操作成功仅表示相应转换成功，不证明模型关系。诊断位置是尽力报告的 claim、point、根 block、最内层 step 和 obligation 索引；未知位置可缺省，诊断不参与数学判断或控制流，统计量也不授予接受权威。

2.9 正确性不变式

对当前完整输入域与路径假设作归纳，保持以下性质：

1. 所有可见局部事实对该域中的每一点成立，且引用解析不越过有效作用域。
2. Lin、Round、EqPair、Plotkin 的规则保持事实有效；Cases 由两穷尽子域推出共同事实。
3. Split 覆盖当前域，Absurd 排除的域确无可行点。
4. 每个 lift 的可信引理保证每个父点都有一个满足全部生成条件的共同扩展点。
5. 每个 Return 对一个共同 Y 证明全部实际输出条件与调用者附加关系。
6. 改进叶在严格根域上要么矛盾，要么输出一个可行且严格下降的共同 Y。

由此每个已验方向建立第一章规定的总存在关系；双射和点合约再依其独立域构造与检查建立各自命题。这是一份规则层面的正确性论证，其可信前提包括精确算术、模型导入与结构验证、codec、作用域实现以及各静态 lift/Plotkin 规则的 Rust 实现。它没有声称源程序已经由证明助手形式化验证。

第三章　操作与推理规则参考

本章逐条定义当前六种 Step、两种 DerivationEnd 与十种 End 的前置条件、精确语义、状态变化及拒绝条件。Kind 为 Le/Eq/Int，Ref 的九种形式与 Exact 规则见第二章。载荷是逻辑字段，不是本章另定义的 opcode 布局；逐字节记录、保留位与版本门见第四章。

3.1　操作总表与共同条件

Step tag	名称	最低版本及作用
0	Lin	V1；产生一种精确线性组合事实。
1	Round	V1；结合 LE 和 INT 得到精确舍入不等式。
2	Drop	V1；删除显式 ID 列表。
3	EqPair	V2；由相反 LE 得到 EQ。
4	Cases	V2；两穷尽局部证明产生一条共同事实。
5	Plotkin	V3；静态二元码距离上界。

End tag	名称	最低版本及作用
0	Return	V1；完整共同输出向量和全部义务。
1	Split	V1；两穷尽完整子 Block。
2	Absurd	V1；恒定矛盾关闭当前域。
3	FlowLift	V4；共同源商品流分解。
4	IntervalLift	V5；同时选择实区间坐标。
5	MenuLift	V7；可选整数菜单的二元展开。
6	SignedLift	V8；有激活的有界实数正负分解。
7	SubflowLift	未冻结 V10；同时保留有需求的有界子流。
8	ImproveReturn	未冻结 V10；同模型完整严格改进向量。
9	PositiveRefutation	未冻结 V10；与严格根条件矛盾。

产生事实的 Step 在成功后将 (kind, e) 发布至显式 `id`。所有表达式必须在当前 input 维度规范，标量及运算中间值必须精确且满足资源界。活跃 ID 不得覆盖；拒绝任何前提引用失活、错误种类或不匹配表达式。Drop 不产生事实，End 不向父作用域发布局部事实。证书没有 NOP、逐项 MAP、单独 CHECK、RET、YIELD 或动态定理调用操作。

3.2　Lin：精确线性组合

字段为 `id`、`kind`、精确 `constant` c 与有序 `terms` (r_k, λ_k)。先解析每个 r_k 为事实 (K_k, e_k)，构造并规范化

$$e = c + \sum_k \lambda_k e_k.$$

允许条件依输出 Kind 分别为：

输出	前置条件
Le	$c \le 0$；每个前提为 Eq（乘子任意有理数），或 Le 且 $\lambda_k \ge 0$。
Eq	$c = 0$；每个前提均为 Eq，乘子任意有理数。
Int	$c \in \mathbb{Z}$；每个前提为 Eq（乘子任意有理数），或 Int 且 $\lambda_k \in \mathbb{Z}$。

语义是对有效不等式做非负组合、对零等式做任意组合，或对整数值做整数组合。INT 中 rational 倍 Eq 仍等于零，不要求该倍数为整数。空 terms 可产生常数不大于零的 LE、零 EQ 或常数 INT。常数不满足对应条件、LE 的负不等式乘子、EQ 的非等式前提、INT 的分数整数乘子均拒绝。**必须：**即使 $\lambda_k = 0$，也须解析并检查该前提的引用和种类；零乘子不隐藏无效引用或 Exact。成功后只发布这一条规范 e；本指令不会自动把正比例相等的事实视为相同。

3.3 Round：整数表达式精确舍入

字段为 `id`、`le` 与 `integer`。两原始引用须分别为

$$\mathrm{LE}(a), \qquad \mathrm{INT}(z), \qquad a = c_a + \sum_i a_i x_i, \quad z = c_z + \sum_i a_i x_i.$$

变量项列表必须精确相同，常数可不同。令 $d = c_a - c_z$，发布

$$\mathrm{LE}\big(z + \lceil d \rceil\big).$$

因为 $a \le 0$ 给出整数 $z \le -d$，故 $z \le \lfloor -d \rfloor = -\lceil d \rceil$。取整作用于精确有理数，对负数也使用数学上取整。前提不是 Le/Int、变量支持或系数不同、引用无效、Exact 以及任何超限中间数均拒绝。上界和下界舍入共用这一个规则；下界须先把相应表达式改为负号形式，没有额外 ROUND.LO opcode，也没有未证整数表达式的自动舍入。

3.4 Drop：局部可见性删除

字段为 `ids`，是一个显式 u32 数组，不是半开寄存器区间。按数组顺序删除各个当前可见 Local ID；空数组有效。删除继承事实时只在当前子作用域隐藏它，父和兄弟仍可见。成功删除后该 ID 可被后续定义复用。引用缺失或已删除的 ID 时拒绝，同一 Drop 中重复 ID 也会因第二次删除不存在事实而拒绝。该指令不删除模型事实或假设，也不修改模型、map 或任何已验数学命题。

3.5 EqPair：精确反对称性

字段为 `id`、`positive` 和 `negative`。两引用须种类均为 Le，表达式分别精确为 e 与 $-e$。发布 $\mathrm{EQ}(e)$，保留 positive 的表达式方向。前提为 Eq、仅按正比例相反、常数不相反、Exact 或引用失活时均拒绝。语义为 $e \le 0$ 且 $-e \le 0$ 推出 $e = 0$；若后续需 $\mathrm{EQ}(-e)$，须用 Lin 显式乘以 -1，不由该操作隐式改向。

3.6 Cases 与 DerivationEnd：局部穷尽证明

字段为 `id`、`kind`、`expression` p、`split` e、可选 `integer`、以及 `left`、`right` 两 Derivation。p, e 在父维度验证。无 integer 时加入 $\mathrm{LE}(e)$ 与 $\mathrm{LE}(-e)$；有 integer 时须原始 Int 引用恰证明 e，加入 $\mathrm{LE}(e)$ 与 $\mathrm{LE}(1 - e)$。

每个子证明先执行自己的 Steps，随后使用独立 DerivationEnd 命名空间：

tag	端点	条件
0	Witness(Ref)	按目标检查规则证明恰为 $(\texttt{kind}, p)$，允许相同 Eq 及合法 Exact。
1	Absurd(Ref)	证明无变量项且常数为正的 Le，或非零常数 Eq；不得使用 Exact。

两支隔离，均成功才在父发布 $(\texttt{kind}, p)$；子内部定义不会泄漏。若两支没有同一指定结论、不合法关闭矛盾、覆盖父活跃 ID、integer 不是准确 Int、任一子证明失败或深度超限，整个 Cases 拒绝。它可以嵌套 Cases，却不能加入输出 map、lift 或严格改进端点。其结论的有效性来自穷尽覆盖，不是把父事实库与两个子事实库合并。

3.7 Plotkin：带已验近对 clique 的码上界

字段为 `id`、`bits` b、`distance` d、`variables` (i, ℓ_i) 和 `cliques`。要求 $1 \le b, d \le 64$ 且 $2d > b$，input integer mode 开启。选中坐标不得重复，实际变量须声明整数、具有有限下界 ≥ 0 与有限上界 ≤ 1。每个 u64 label 只能使用低 b 位；$b = 64$ 时全部位可用。label 不要求互异，但相同 label 的选中变量必须由近对前提约束。

每个 clique 原始引用必须是严格如下形式的 Le：

$$\mathrm{LE}\left(\sum_{i \in C} x_i - 1\right),$$

所有出现系数均为 $+1$，常数为 -1，每个出现坐标的实际有限下界不负。clique 可含未选中的其他非负坐标，它们不要求二元。对选中变量的每一对 (i, j)，若 $\mathrm{popcount}(\ell_i \operatorname{xor} \ell_j) < d$，至少一个已验 clique 必须同时包含两者。clique 错种类、错误常数/系数、负下界、Exact 或任何未覆盖近对均拒绝。

发布

$$\mathrm{LE}\left(\sum_{i \text{ selected}} x_i - \left\lfloor \frac{2d}{2d - b} \right\rfloor\right).$$

证明：在任意整数可行点，激活的 k 个二元变量的 labels 两两距离至少 d。总两两距离至少 $dk(k-1)/2$；按位计，每位贡献至多 $k^2/4$，所以总距离至多 $bk^2/4$，推出所示界（$k = 0, 1$ 也成立）。这是有限静态可信推理规则，不依据图名称或外部求解结果。未选择变量或没有 clique 也按相同检查执行，近对不存在时覆盖要求为空真。配对数量、clique 成员位表和扫描工作另受资源界约束。

3.8 Return、Split 与 Absurd

3.8.1 Return：完整输出

字段为 `map` 和 `checks`。仅普通方向上下文可用。map 长度恰为实际输出维度，所有表达式使用当前输入坐标并规范。checks 恰按第2.5节完整义务顺序逐项证明。成功即完成该叶的完整存在构造，不产生继续执行的位置。缺失/多余 map、错误维度、缺失/多余 checks、任一义务不匹配或错误上下文均拒绝。输出条件不得用不同的 map 元组分别证明。

3.8.2 Split：穷尽完整子 Block

字段为 `expression` e、可选 `integer`、`left`、`right`。表达式与整数引用条件和 Cases 完全相同。两子 Block 继承父事实、各加入一条分支假设，并依序全部执行。每个子体须成功完成当前上下文允许的完整端点。成功表示每个父点至少属于一个已完成子域；不合并 map 或部分检查项。引用/表达式错误、非 Int 整数前提、失败子体或深度超限均拒绝。没有专门的二元枚举、有限区间或多路 split opcode；需要时可用规范 e 与二叉树表达。

3.8.3 Absurd：恒定矛盾

字段为 `fact`。解析为当前事实后，要求表达式变量项为空，且为 LE(c)、$c > 0$ 或 EQ(c)、$c \neq 0$。成功关闭当前域的空支，不生成输出向量。Int、非恒定式、非矛盾常数、失活引用和 Exact 均拒绝。此操作可在普通方向、改进和 LP 反证中使用，DerivationEnd Absurd 使用同一矛盾谓词。

3.9 共同扩展的公共约定

五种 lift 均为 End，载荷含 witness 和一个 body Block。记进入端点时变量数为 N、行数为 R。每个 witness 表达式只能使用这些旧坐标；全部组的全部前提先在旧模型与旧事实作用域检查，才允许 body 访问生成模型。原模型名称、变量/行向量与目标保持完整前缀。新名称避碰，不决定编号。对生成行的引用遵守普通 RowLower/RowUpper/RowEq 条件。

各 builder 检查 count 加乘、端点编号、规范表达式、有理数中间值、扩展维度与项数，并在剩余请求预算内执行 fallible allocation 与名称生成。witness 预备成功只得到候选模型与待证前提；不表示前提已经有效。任何前提错误或作用域错误、legacy inverse 根约束、禁止 lift 的反证上下文、模式/深度/资源超限均拒绝。一旦进入 body，全部新变量和全部新行可同时使用，原事实和路径假设继续有效。

以下存在论证覆盖实数输入点，不只覆盖可枚举的有理样本。生产 checker 构造公式，不执行这些数学论证中的搜索或数值选择。

3.10 FlowLift：共同源商品分解

字段为 `witness`、`body`。FlowWitness 给出 `nodes`、有序 `arcs` (tail, head) 和非空 `groups`。节点数 $n \geq 1$，所有图端点在 $[0, n)$；允许平行弧、环和零弧。每组给出 `source` s、非空 `commodities` (t_k, d_k)、每弧的 `flows` F_e、`nonnegative` 引用和 `balances` 引用。每个 $d_k \geq 0$；允许重复终点、$t_k = s$ 和零需求。flows/nonnegative 长度必须恰为弧数 m，balances 恰为节点数 n。

定义散度为出流减入流，环在散度中抵消；令

$$b_v = \sum_k d_k(\mathbf{1}_{v=s} - \mathbf{1}_{v=t_k}).$$

每组按序要求先所有弧 LE($-F_e$)，然后所有节点

$$\mathrm{EQ}\left(\sum_{e:\,\mathrm{tail}=v} F_e - \sum_{e:\,\mathrm{head}=v} F_e - b_v\right).$$

零式和冗余节点仍须提供有效引用。

每组按 commodity、arc 顺序追加连续 $z_{k,e} \geq 0$，无有限上界。若本组起始坐标为 S，则 $z_{k,e}$ 的编号为 $S + km + e$。先按 commodity、node 顺序追加

$$\mathrm{div}(z_k)_v = d_k(\mathbf{1}_{v=s} - \mathbf{1}_{v=t_k}),$$

再按 arc 顺序追加 $\sum_k z_{k,e} - F_e = 0$。每组 K 个 commodity 增加 Km 变量、$Kn + m$ 行。商品平衡行常数零，右侧为两个相等界；弧总量行保留 $-F_e$ 的完整常数、两界零。空弧图仍保留全部商品节点等式。

共同存在。 固定满足前提的父点，从残余非负 F 逐商品提取正残余的简单源到终点路径。未满足的终点必可到达：否则能到该终点的正残余可达集合不含源、没有正残余入弧，总散度应非负，而剩余终点需求给出负总散度，矛盾。每次减去路径瓶颈与剩余需求的较小值；一次完成一个商品或永久消去一弧，有

限步终止。零需求或源等于终点无需路径。最后残余是环流，将全部残余加入第一个商品，保留所有弧总量，含自环及断开的环流。各组新坐标互不相交，故选择共同存在。

这不授予每商品上界、整数性或异质成本关系；生成 z 全为连续。缺失商品、负需求、图端点错误、错误前提数或旧维度之外 F 均拒绝。输出中的额外容量、目标与投影必须在 body 另证。

3.11 IntervalLift：独立实区间选择

字段为 `intervals`、`body`。每项 IntervalWitness 为有序 `lower` L_i、`upper` U_j 与 `checks`。checks 长度须等于两列表长度的精确乘积，依 lower-major 次序证明

$$\mathrm{LE}(L_i - U_j) \quad \text{对每个} (i, j).$$

无 lower 或无 upper 时不产生配对前提。所有项可共享父坐标，但不能引用同一次 lift 新增的坐标。

按项序追加一个无固有有限界的连续坐标 z，第 k 项编号 $N + k$。行按项序，先 lower 列表的 $z - L_i$ 下界零行，再 upper 列表的 $z - U_j$ 上界零行；另一侧缺失，表达式保留负旧仿射常数。自由区间仍追加变量；空 intervals 是恒等扩展。

共同存在。 有 lower 时取其有限列表最大值，配对前提保证它不超过所有 upper；无 lower 有 upper 时取 upper 最小值；两列表皆空取零。每项只依赖同一固定父点，所有值可同时选取。规则不产生整数 z，也不自动赋予不同区间间的耦合条件。错误检查数量/顺序/表达式、非法旧坐标、配对乘积或中间差超限均拒绝。

3.12 MenuLift：可选二元菜单

字段为 `menus`、`body`，须 input integer mode。每 MenuWitness 含 u32 `maximum_index` K、`activation` A、`index` I，以及恰六个 checks，依序证明

$$\mathrm{LE}(-A),\ \mathrm{LE}(A-1),\ \mathrm{INT}(A), \qquad \mathrm{LE}(-I),\ \mathrm{LE}(I-KA),\ \mathrm{INT}(I).$$

A, I 是旧坐标规范仿射式，不要求原本就是单个整数坐标，但必须证明其整数值。

每菜单按 $j = 0, \ldots, K$ 追加 $K + 1$ 个声明整数且界为 $[0, 1]$ 的 z_j，然后依序追加两行、两界均零：

$$\sum_{j=0}^{K} z_j - A = 0, \qquad \sum_{j=0}^{K} j z_j - I = 0.$$

$K = 0$ 仍有两行，第二行是 $-I = 0$，不保存 z_0 的零系数。空菜单列表是恒等扩展，但整数模式与深度要求仍适用。

共同存在。 前提使 $A \in \{0, 1\}$。$A = 0$ 时 $I = 0$，取全零；$A = 1$ 时 $I \in \{0, \ldots, K\}$，只取 $z_I = 1$。所有菜单独立依父点构造，共同可行且各自菜单向量唯一。此局部唯一性不建立完整模型双射；仍须独立完整域单射证明与全输出组合。关闭 input integer mode、缺失六前提、不可证明 A/I 整数、尺寸或 KA 中间值超限均拒绝。

3.13 SignedLift：有激活的有界实数正负分解

字段为 `decompositions`、`body`，须 input integer mode。每 SignedWitness 含精确非负 `capacity` C、旧式 `flow` F、`activation` Z，以及恰五个 checks，依序证明

$$\mathrm{LE}(-Z),\ \mathrm{LE}(Z-1),\ \mathrm{INT}(Z), \quad \mathrm{LE}(F-CZ),\ \mathrm{LE}(-F-CZ).$$

不假设 F 整数。每项按 p,n,u,v 顺序追加四坐标：p,n 连续且非负、无有限上界；u,v 声明整数、界为 $[0,1]$。每项按序追加四行：

$$p-n-F=0,\quad u+v-Z=0,\quad p-Cu\le 0,\quad n-Cv\le 0.$$

前两为两界零的等式；后两仅上界零。$C=0$ 保留容量行但省去零系数；空列表是恒等扩展，仍检查模式和深度。

共同存在。 $Z=0$ 时前提强迫 $F=0$，取全零。$Z=1,F\ge 0$ 时取 $(F,0,1,0)$，否则取 $(0,-F,0,1)$。容量前提保证全部界；当 $F=0,Z=1$ 时仍保持 $u+v=1$，不能删除激活。所有项依同一父点同时选择。负/非规范 C、缺失五前提、错误模式、旧维度攻击及 CZ 中间值超限均拒绝。规则不自动证明实际目标、输出 side constraints 或完整双射。

3.14 SubflowLift：有需求部分的共同有界子流

未冻结 V10。字段为 `witness`、`body`。SubflowWitness 的 `nodes`、`arcs`、非空 `groups` 与 Flow 的图条件相同。每组含 `source` s、有序 `demands` (t_i,d_i)、每弧 `flows` F_e、`nonnegative` 和 `balances`。需求列表可空，重复终点可用，$d_i\ge 0$；每个终点都须不同于源，包括零需求。flows/nonnegative 必须恰为 m 项，balances 恰为 n 项。令 $B=\sum_i d_i$，$b_s=B$，其他 $b_v=-\sum_{t_i=v} d_i$。依组、弧、节点顺序证明 $\mathrm{LE}(-F_e)$ 和 $\mathrm{EQ}(\mathrm{div}(F)_v-b_v)$。

每组按弧序追加 m 个连续坐标 g_e，有限界为 $[0,B]$。先按 node 序加 n 个等式 $\mathrm{div}(g)_v=b_v$，再按 arc 序加 m 个 $g_e-F_e\le 0$。前者零常数、两侧为 b_v；后者保留 $-F_e$ 常数且只上界零。G 组共加 Gm 变量、$G(n+m)$ 行。组 a 的变量起点 $N+am$，行起点 $R+a(n+m)$。零弧和空需求合法，仍保留每组所有节点等式。

共同存在。 $B=0$ 取 $g=0$。否则加入一个超级终点及每个正需求的终点到超级终点弧，流量为该需求。增广非负流只有源散度 B、超级终点散度 $-B$。从源提取正残余的简单路径到超级终点，减去瓶颈并在旧弧累加到 g。只要源尚有正残余散度，超级终点必可达：否则源的正残余可达集合无正出弧、总散度应不正，却等于正的剩余需求，矛盾。每次永久消去至少一弧，有限步完成；路径权重总和为 B，每简单路径至多使用某弧一次。故同一 g 满足 $0\le g_e\le F_e$、$g_e\le B$ 和全部需求散度。剩余环流丢弃，自环与断开的圈无需另加条件。各组使用互不相交的新坐标，故共同存在。

它不授权对每弧独立截断：独立 clip 可能破坏流守恒。非法终点、源等于需求终点、负需求、缺失前提、旧维度攻击及预算超限均拒绝。额外 side constraints 与目标关系必须由 body 完整证明，不能从子流引理猜得。

3.15 ImproveReturn 与 PositiveRefutation

3.15.1 ImproveReturn：同模型严格下降

未冻结 V10。字段为 `map`、`checks` 和 `margin`，只在改进上下文可用。map 给出同一实际完整模型的全部输出坐标。margin 为 `Constant`(q) 或 `Predicate`(q)，要求精确规范 $q>0$。依普通变量/行顺序证明全部可行性，随后证明唯一目标不等式

$$f_M[Y]-f_M(x)+q\le 0\quad 或\quad f_M[Y]-f_M(x)+qP(x)\le 0.$$

根域 $P>0$ 保证严格下降。checks 必须恰好，无 projection/inverse 等附加项。普通方向使用、零/负 margin、不完整共同 Y、漏验某行或义务不匹配均拒绝。其成功不证明最优值存在，不为改进树以外的事

实库输出严格不等式。

3.15.2 PositiveRefutation：与严格根条件矛盾

未冻结 V10。字段为 `fact`，只在 Improvement Block 中可用。按目标命题检查精确证明 LE(P)；允许已有同式 Eq 或合法 Exact。与不可变根条件 $P > 0$ 矛盾，因而关闭此严格子域。P 使用原始坐标，其含义不受 lift 改变。普通方向、LP/fiber 反证、Cases Derivation 或错误表达式/种类均拒绝。它不需要证明变量项为空，但这个例外只适用于所绑定的 P，不能借此任意关闭非恒定不等式，也不声明弱域 $P \geq 0$ 为空。

第四章　MCM 与 RPX 二进制格式

本章给出当前 `codec.rs` 可接受并可输出的逐字段布局。MCM 只有版本 1；RPX 接受版本 1–10，其中 V1–V8 已冻结，V9 与 V10 是已实现、未冻结的工作草案。旧 D0 的 128 B header、section directory、ScalarPool、ExprId、对齐 tape 和隐式 FactId 均不属于当前协议。实现者应按本章的顺序记录编码，不得把旧提案与当前记录混合。

4.1　通用记法与文件头

`u8/u16/u32/u64` 是固定宽度无符号整数；所有多字节整数采用 little endian。串接符号 `；`只表示字段次序，不写入文件。记录没有隐式 padding、宿主结构体布局、指针、浮点数、对齐间隙或独立长度头。读取一个嵌套记录时，其子字段和计数决定记录终点。任何偏移、计数和长度运算必须检查溢出，读取必须先验证剩余范围。

MCM 与 RPX 的文件头均为 16 B：

偏移	宽度	字段	要求
0	4	magic	MCM 为 `4D 43 4D 00`；RPX 为 `52 50 58 00`。
4	2	version	MCM 必须为 1；RPX 必须为 1–10。
6	2	reserved	必须为零；不是能力位。
8	8	file length	必须等于实际提供的完整字节数组长度，包括本头。

header 不含 source/target 摘要或 contract word。两个实际模型和独立 Request 由调用接口提供；模型绑定与请求一致性见第五章。未知版本、未知记录 tag、保留字段非零、截断记录以及解析结束后的尾随字节均拒绝，当前接口把不支持的版本也报告为 `INVALID`，没有单独的 `UNSUPPORTED` 状态。

以下记法在后续所有表中具有固定意义：

记法	字节布局
`tag(t)`	`u8 t; u8 0; u16 0`，共 4 B。所有三个保留字节必须为零。
`array<T>`	`u32 count`，然后 count 个紧密连接的 T，数组本身无结尾符。
`option<T>`	单个 `u8`：0 表示无后续值；1 表示紧接一个 T；其他值拒绝。
`name`	`u32 byte_count`，随后恰好该长度的 UTF-8 字节，无 NUL 结尾。模型验证另要求 ASCII、非空、无控制字节、至多 1,024 B。
`Q`	下节的规范任意精度有理数。
`Expr`	`Q constant; array<(u32 variable,Q coefficient)>`。
`Steps`	`array<Step>`。

不同 enum 使用各自的 tag 命名空间：Step 的 0、End 的 0 和 PointClaim 的 0 不是同一记录。`option` 也不使用四字节 `tag`。

4.2 规范有理数与表达式

4.2.1 Q 的布局与拒绝规则

一个 Q 的固定前缀为 12 B：

Q 内偏移	宽度	字段	含义
0	4	`tag(sign)`	0 为非负；1 为负；其他值拒绝。
4	4	numerator length	分子绝对值字节数，可以为零。
8	4	denominator length	分母字节数，必须非零。
12	变长	numerator magnitude	无符号 little-endian，长度由分子长度给出。
随后	变长	denominator magnitude	无符号 little-endian，长度由分母长度给出。

非空 magnitude 的最后一个字节不得为零，故不允许冗余高位零。分母为严格正整数，分子与分母必须互素；decoder 先构造精确比值，再比较约分前后分子、分母，遇到 `2/4` 会拒绝，不能在 wire 层自动修复。零只有一种编码：sign=0、numerator length=0、denominator length=1、denominator=1。负零、分子用单字节零、`0/7`、零分母、非最短 magnitude 均拒绝。每个分子或分母 magnitude 当前至多 1,700 B。

整数也使用 Q：非零整数的分母是单字节 1，没有内联整数或 scalar index。例如 $-1/2$ 的完整 Q 字节是：

```
01 00 00 00  01 00 00 00  01 00 00 00  01 02
```

前 4 B 是负号 tag，随后两个长度均为 1，最后两个字节分别为分子、分母 magnitude。Q=0 占 13 B，其他最小非零 Q 占 14 B。

4.2.2 Expr 的布局

Expr 先写完整 Q 常数，再写 `u32 term_count`，每项先写 `u32 variable` 再写完整 Q 系数。variable 必须严格递增，不能重复；每个系数必须非零。零表达式是 Q=0 加 term count=0，占 17 B。常数项不是编号为零的变量项。所有表达式直接存入使用处，没有共享 expression pool。

codec 检查项顺序和非零性；在有具体模型的验证阶段还要检查 variable 的有效范围。模型内表达式使用本模型坐标；普通 Block 表达式使用当前输入坐标，含已准许的 lift 坐标。Claim 的 affine observations、halfspace 以及 point 的 map 按各自 contract 使用原始坐标。这些语义不能由一个合法 u32 编码取代。

4.3 MCM 1 模型布局

文件头之后按下列顺序直接连接：

```
name model_name;
array<Var> variables;
array<Row> rows;
Expr objective
Var = name; tag(integer); option<Q> lower; option<Q> upper
Row = name; Expr expression; option<Q> lower; option<Q> upper
```

Var 的 domain tag 仅允许 0（连续）和 1（整数）。binary 已表现为整数标志与实际 bounds，没有独立 binary tag。没有 bound 的一侧表示对应的无限方向，而不是特殊 Q。objective 始终是完整仿射最小化目标，包含常数。变量和 row 均按数组下标编号；变量名在变量集合内唯一，row 名在 row 集合内唯一。两集合之间允许同名。模型名称、变量名称和 row 名称都参与实际模型数据。

每个模型表达式的 variable 必须小于 variable count。有限 lower 大于 upper 允许被编码并验证为模型结构；它表示空可行集。零变量、零 row、常数 row 和常数目标均允许。MCM decoder 在读完且确认无尾随字节后调用完整 `Model::validate`。导入时的命名、排列、sense 与 bounds 规则见第五章。

4.4 RPX 外层与 Claim

header version 决定外层字段是否存在。存在但为空的数组仍写四字节零计数；版本不包含的数组没有任何字节，decoder 将它补成内存中的空数组。

次序	字段	最低版本
1	`array<Claim> claims`	1
2	`array<Block> blocks`	1
3	`array<PointWitness> points`	3
4	`array<BijectionWitness> bijections`	7
5	`array<ImprovementWitness> improvements`	10

claims 与 blocks 计数必须相等，且第 i 个 Block 证明第 i 个 Claim。空 certificate 在语法层可以解码，但不能通过空 Request 的数学验证。

每个 Claim 的固定前缀为 `u8 flags; u8 objective; u16 0`。flags 位 0 表示 reverse，位 1 表示 input integers，位 2 表示 output integers；V10 的位 3 表示存在 input halfspace。位 4–7 保留为零；V1–V9 也要求位 3 为零。objective byte 为 0 None、1 Equal、2 NonIncreasing；其他值拒绝。前缀之后的字段顺序如下：

字段	布局及含义	最低版本
projection	`array<(u32 input,u32 output)>`；按请求顺序的坐标相等观察。	1
inverse	`option<u32>`；引用 claims 中的 opposing Claim 下标。	1
input objective lower	`option<Q>`；输入完整目标的请求下界。	3
affine projection	`array<(Expr input,Expr output)>`；两端各在原始模型坐标中。	5
input objective level	`option<Q>`；实际输入完整目标等于该值的条件域。	9
input halfspace	位 3 为 1 时恰好一个 Expr，紧随 objective level；没有额外 option byte。	10

版本低于对应行时整个字段缺席；半空间仅由 flags 位控制。不能为“未来字段”补零，否则它们会被解析为后续字段或者尾随字节。半空间表达式 H 表示输入条件 $H(x) \le 0$，不能把 `RowUpper` 自动提升成 equality。Claim 的完整数学意义和 return obligations 见第一章及第二章。

4.5 Ref、Kind 与 Step

Ref 是独立四字节 tag 后接可选的 u32 ID；没有 packed 高位 tag。

tag	Ref 名称	tag 之后的字段与约束
0	VarLower	u32 variable；必须有有限 lower。
1	VarUpper	u32 variable；必须有有限 upper。
2	VarInt	u32 variable；实际 integer 且本方向启用 input integers。
3	RowLower	u32 row；必须有有限 lower。
4	RowUpper	u32 row；必须有有限 upper。
5	RowEq	u32 row；两侧必须有限且精确相等。
6	Local	u32 显式 fact ID；必须当前可见且 live。
7	Assumption	u32 假设数组下标；外层先于内层编号。
8	Exact	无 ID，共 4 B；只可在要求检查一个确切表达式的 obligation 位置使用。

0–7 的 Ref 各占 8 B；8 以外 tag 拒绝。Assumption(0) 是最外层假设，后入栈假设取更大的 index；它不是 “从当前向外数零层”。V10 improvement 的弱根假设占 index 0，随后 Split/Cases 假设从 1 开始。Exact 不是环境中的事实，不能作为 Lin、Round 或普通矛盾引用。

Kind 也采用 `tag`：0 Le、1 Eq、2 Int；其他值拒绝。Step 的字段按表中顺序编码，表内所有 id/bits/distance/variable 均为 u32，label 为 u64。

tag	名称	tag 之后的布局	最低版
0	Lin	`id; Kind; Q constant; array<(Ref fact,Q multiplier)> terms`	1
1	Round	`id; Ref le; Ref integer`	1
2	Drop	`array<u32> ids`	1
3	EqPair	`id; Ref positive; Ref negative`	2
4	Cases	`id; Kind; Expr expression; Expr split; option<Ref> integer; Derivation left; Derivation right`	2
5	Plotkin	`id; bits; distance; array<(u32 variable,u64 label)> variables; array<Ref> cliques`	3

producer 的 id 是作者提供的显式 Local ID，没有隐式结果编号或 out ExprId。不能覆盖 live ID；Drop 删除当前作用域中的可见性后可以重用该 ID。Drop 的数组不是 Ref 数组，也不携带记录 tag。Lin terms 与 Expr terms 不同：前者项的第一字段为 Ref。codec 不代替语义检查：系数的符号、事实类型、scope 和未来引用等须按指令章检查。

4.6 Block 与 Derivation

`Block = Steps; End`；`Derivation = Steps; DerivationEnd`。二者都在 Steps 后恰好有一个 terminal，没有独立 byte length、pc 或 continuation。DerivationEnd 的 tag 0 是 `Witness; Ref`，tag 1 是 `Absurd; Ref`；它只结束局部 Cases 推导，不能提供输出 map。

End 的 tag 与字段如下。所有子 Block 按出现次序完整编码。

tag	名称	tag 之后的布局	最低版
0	Return	`array<Expr> map; array<Ref> checks`	1
1	Split	`Expr expression; option<Ref> integer; Block left; Block right`	1
2	Absurd	`Ref fact`	1
3	FlowLift	`FlowWitness witness; Block body`	4
4	IntervalLift	`array<IntervalWitness> intervals; Block body`	5
5	MenuLift	`array<MenuWitness> menus; Block body`	7
6	SignedLift	`array<SignedWitness> decompositions; Block body`	8
7	SubflowLift	`SubflowWitness witness; Block body`	10
8	ImproveReturn	`array<Expr> map; array<Ref> checks; ImprovementMargin margin`	10
9	`Positive Refutation`	`Ref fact`	10

Return 的 map 必须在 output coordinate 顺序中为每个变量提供一个 Expr。checks 的长度和顺序由实际 output model 与请求生成的全部 obligations 决定，不能只提供部分检查。Split 两个分支分别完成整组 obligations 或矛盾，没有不完整结果的 merge。ImproveReturn 只用于同一实际模型的 strict improvement，普通 Return 不能替代它；PositiveRefutation 也只在 improvement context 合法。能被 wire 解码不等于能在任意语义 context 使用。decoder 即使面对语义上不可行的分支也完整读取其所有记录并执行版本检查。

4.7 联合扩展 witness 的逐字段布局

所有 witness 使用调用前同一个输入 scope；其 checks 不使用刚生成的变量或 row。这些记录不自行证明存在性，所调用的静态数学 lemma 属于可信边界。

记录	依次编码的字段
FlowWitness	`u32 nodes; array<(u32 tail,u32 head)> arcs; array<FlowGroup> groups`
FlowGroup	`u32 source; array<(u32 sink,Q demand)> commodities; array<Expr> flows; array<Ref> nonnegative; array<Ref> balances`
IntervalWitness	`array<Expr> lower; array<Expr> upper; array<Ref> checks`
MenuWitness	`u32 maximum_index; Expr activation; Expr index; array<Ref> checks`
SignedWitness	`Q capacity; Expr flow; Expr activation; array<Ref> checks`
SubflowWitness	`u32 nodes; array<(u32 tail,u32 head)> arcs; array<SubflowGroup> groups`

记录	依次编码的字段
SubflowGroup	`u32 source; array<(u32 sink,Q demand)> demands; array<Expr> flows; array<Ref> nonnegative; array<Ref> balances`

Flow 与 Subflow 的图数组元素恰好 8 B。每个 demand 元素是 u32 sink 加完整 Q。它们在 wire 上可以出现任意 u32 endpoint，语义 checker 再核对 nodes 范围、维数与排列。interval 的 lower 与 upper 数组可以为空，表示自由或单侧 real choice。Menu 必须按 $-A, A-1, \mathrm{INT}(A), -I, I-KA, \mathrm{INT}(I)$ 顺序提供六个 checks；Signed 按 $-Z, Z-1, \mathrm{INT}(Z), F-CZ, -F-CZ$ 提供五个 checks。Flow、Interval、Subflow 的完整生成布局和 premise order 见第三章，不能用“同样数量的任意事实”替代规定表达式。

4.8 Point、Bijection 与 Improvement

PointWitness 为 `PointClaim; array<Q> values`；V6 及以上追加 `option<Block> refutation`。V3–V5 完全没有该 option byte。PointClaim tag 后的布局如下：

tag	名称	字段	最低版
0	Feasible	`u8 flags; u8 0; u16 0; option<Q> objective`；位 0 选 target，位 1 启用 integers，其他位为零。	3
1	Strictness	`u32 inclusion_claim`	3
2	FiberStrictness	`u32 inclusion_claim`	6
3	ImageExclusion	`u8 flags; u8 0; u16 0; array<Expr> map`；位 0 是 point in target，其他位为零。	10

Feasible 的两个四字节部分不能合并：第一个是 enum tag，第二个是它自己的 flags/reserved。values 要覆盖所选实际模型的全部坐标。FiberStrictness 必须带 refutation；Feasible、Strictness 和 ImageExclusion 必须没有 refutation，surplus block 即使格式合法也拒绝。ImageExclusion 的 map 是调用者指定、由所选实际模型原始坐标到另一实际模型全部坐标的仿射 tuple。它在 LP mode 检查实际违例，不能靠整数 fractional value 宣布排除。

BijectionWitness 没有额外 tag，布局为：

```
u32 forward_claim; u32 reverse_claim;
Steps injectivity_steps; array<Ref> injectivity_checks
```

两个索引分别指同一 claims 数组中的相反、兼容方向。injectivity 没有 End；其 Steps 后直接是 equality checks 数组。完整成对输入域构造和 composition obligation 不能被局部 menu 唯一性代替。

ImprovementWitness 也没有额外 tag，布局为：

```
u8 flags; u8 0; u16 0; Expr positive; Block block
ImprovementMargin = tag(0); Q   // Constant(q)
                  | tag(1); Q   // Predicate(q)
```

flags 位 0 选 target，位 1 启用 integers，其他位为零。margin 的 Q 在数学检查中必须严格为正；常数形式表示固定下降量 q，predicate 形式表示 $qP(x)$。该 contract 的根条件是 $P(x) > 0$，没有 wire 上的 strict-LE fact kind。

4.9 版本选择、降版与资源预检

encoder 总是选择证书实际需要的最低版本，而不是保留原输入 header。扫描遍历普通 Block、Cases 的 Derivation、point refutation、bijection injectivity 以及全部 improvement Block；不可行子分支也必须扫描。

最低版	触发条件
1	Lin/Round/Drop、普通 Return/Split/Absurd、基础 Claim。
2	任意 EqPair 或 Cases。
3	Plotkin、非空 points、input objective lower。
4	FlowLift。
5	IntervalLift 或非空 affine projection。
6	FiberStrictness 或任何 present point refutation。
7	MenuLift 或非空 bijections。
8	SignedLift。
9	present input objective level；工作草案。
10	present input halfspace、ImageExclusion、非空 improvements、SubflowLift、ImproveReturn 或 PositiveRefutation；工作草案。

decoder 接受支持范围内高于最低需要的 version，但按该 header 读取全部版本字段。例如 V10 的基础证书仍须包含零计数 points、bijections 和 improvements，以及各 Claim 的 lower/affine/level 字段。它反汇编后再汇编可能降为 V1；数学数据保持相同，header 和因版本而存在的空字段可以改变。因此不能承诺任意可接受高版 bytes 都逐字节 round-trip。把使用新 tag 的文件 header 直接改成旧版会拒绝；它不是合法的降版方法。

当前 codec 限制是实现资源策略：文件至多 1 GiB；单个 counted array 至多 10,000,000 项；所有 counted arrays 累计至多 20,000,000 项；proof depth 至多 64，根 Block/Derivation/Equality depth 计为 1，进入 Cases child、Split child 或 lift body 加 1。name 的 codec 预检上限 1 MiB 不放宽模型验证的 1,024 B 限制。count 先检查累计预算、可表示性和最小剩余字节，再分配；这些最小值是防御性下界，不是实际记录长度，后续每个字段仍需完整读取。

counted 元素类别	每项剩余字节预检下界
Expr term 或 Lin term	17
Steps、graph arcs、coordinate projection	8
Drop ID、Ref array	4
MCM Var / MCM Row	10 / 23
Claim / Block / PointWitness	9 / 12 / 12
Expr array、sink-demand array	17
affine observation pair	34
FlowGroup 或 SubflowGroup	20
IntervalWitness / MenuWitness / SignedWitness	12 / 42 / 51
point values Q / BijectionWitness / ImprovementWitness	13 / 16 / 33
Plotkin variable-label pair	12

codec 格式错误返回 `INVALID`，预算耗尽返回 `RESOURCE_LIMIT`。CLI 的文件读取上限更小，数学模

型、数值运算和 theorem 也有独立限制；详见第五章。全部字节必须解码完成才能执行，不能以分支不可达、未知 optional section 或“只供调试”为理由跳过未经解析的逻辑记录。

本章的直接实现依据是 `crates/core/src/codec.rs` 中 Reader、Writer、`encode_model`、`decode_model`、`certificate_version`、`encode_certificate` 与 `decode_certificate`；类型定义在 `proof.rs` 和 `model.rs`。V10 增量以本章已列出的 halfspace flags 与记录为准。

第五章　Loader、接口与可信边界

一次验证需要调用者选定的完整 source MCM、完整 target MCM、RPX certificate 和独立 Request。当前 RPX 不携带 source/target hash；验证直接读取、检查和使用这两个实际模型。调用者有责任把正确文件或字节绑定到这两个参数，并在调用之后按 Request 的含义解释结果。不可信 certificate 只能提交 proof，不能自行改写调用者请求。

5.1　入口、处理顺序与判定

推荐集成入口为 crate `kernel_core` 的 `api` 模块；`main.rs` 只解析参数、打印 façade 返回的 JSON 和设置 exit code。文件入口先以受限 owned read 获得完整字节，之后依次执行：

1. 对 source 与 target 调用 `codec::decode_model`，包括全部模型结构验证。
2. 调用 `codec::decode_certificate` 完整解析 RPX；任何嵌套分支均不能略过。
3. 调用 `rpa::parse_request` 解析独立请求。
4. 核对 certificate 与 Request 的全部 claims、point claims、bijection claims 及 improvement claims 在数量、顺序与每个字段上精确相等。
5. 用实际模型验证各方向及其 complete Returns；随后验证 bijection injectivity、point witnesses，最后验证 improvements。所有阶段共享同一次工作预算。
6. 只有全部请求通过后返回 `VERIFIED`。解析、导入或装配成功不具有该含义。

模型导入可以事先完成。MPS/LP 不在 `verify` 内重复解析，也没有 solver 回调。

状态	exit	含义
`VERIFIED`	0	独立 Request 的全部数学合同已由此 kernel 接受。
`CANONICALIZED`	0	MPS/LP 解析、模型检查与 MCM 编码完成。
`ASSEMBLED`	0	strict RPA 解析与 RPX 编码完成。
`DISASSEMBLED`	0	RPX 解码并输出 RPA。
`INSPECTED`	0	MCM 解码并报告规模。
`EXPORTED`	0	MCM 解码并导出规范模型 JSON。
`INVALID`	2	格式、模型、请求匹配、证明或数学义务不满足要求，亦包括不支持的版本/方言。
`RESOURCE_LIMIT`	3	已公布预算耗尽；不是验证成功，也不是数学反驳。
`IO_ERROR`	4	文件读写失败。

没有 exit 1 的规定状态，也没有单独的 `UNSUPPORTED`。调用者必须看 status，不能把辅助命令的 exit 0 当成模型关系成立。`INVALID` 也不统一表示“目标数学命题为假”：格式错误和 importer 能力限制都属于该状态。

5.2 可信精确模型导入

5.2.1 共同规范化与模型身份

MPS importer 位于 `model.rs`，LP importer 位于 `lp.rs`；两者和 MCM codec 都是可信代码。它们用任意精度整数与精确有理数进行解析及规范化，不经过浮点数，不调用优化器，不修改原始文件。每个 row/目标表达式把同一变量系数精确相加，按 coordinate ID 排序并删除精确零。零系数的变量出现仍声明变量，不能因为最后项取消而删除该变量。名称、原始声明次序、所有变量/row、bounds、integer flags 和完整目标均保留于规范模型。

所有内部目标均最小化。maximization 使整个仿射目标取负，常数也要取负。缺失 bound side 是真正的无限方向；不能用大有限数代替。互相矛盾的有限 bounds 是空模型的合法表示，不能为了得到可行输入而“修复”。integer 的有理 bounds 不自动取整，因为 LP relaxation 的含义也须保留。模型结构合法不证明模型可行，更不证明 source/target 等价。

5.2.2 MPS 的 sections、cards 与排列

MPS headers 必须从第一列开始，data cards 缩进。次序为 NAME，可选 OBJSENSE/OBJNAME，ROWS，COLUMNS，可选 RHS、RANGES、BOUNDS，最后必须 ENDATA。OBJSENSE 与 OBJNAME 各至多一次；single value 可放在 header 或下一张缩进 card。senses 接受 MIN/MINIMIZE、MAX/MAXIMIZE，默认 MIN；headers、keywords、sense 区分大小写。NAME 未给名称时使用 `MODEL`。

必须恰好有一个 N objective row，E/L/G 是 constraint rows。OBJNAME 若存在必须指定唯一 N row。constraint 顺序为 ROWS 顺序，除去 objective row；variable 顺序为首次 COLUMNS 出现顺序，重复 column cards 必须连续。每个 coefficient card 含一或两组 row/value。重复 coefficient 可相加，重复 row name、未知 row/variable、非连续 column 声明均拒绝。

本方言接受空白分隔 free MPS，以及名称不含空白的普通 fixed-column cards。`COLUMNS/RHS/RANGES` 的 fixed continuation 只有前十四个字符全为空格时才能省略 column/vector name，而且此前须存在可继续的同类 card。fixed-format 的 embedded-space 名称拒绝；不能靠对齐猜测缺失必要字段。第一列 `*` 开始注释；独立 token 以 `$` 开始时截断后续 inline comment，附着在名称中的 `$` 不会截断数据。因此本方言不接受以 `$` 开始的数据名称。空白行和注释行忽略；ENDATA 后非注释内容拒绝。

RHS、RANGES、BOUNDS 每种至多一个 named vector；不默选第一个向量。同一 row 的重复 RHS 或 range card 拒绝。缺失 RHS 是精确零。objective row 的 RHS 值 b 生成 objective constant $-b$，然后才按 sense 归一化；objective 上不能有 range。

5.2.3 MPS bounds、markers 与 ranges

连续 column 默认 $[0, +\infty)$。`'MARKER' 'INTORG'` 至 `'MARKER' 'INTEND'` 的 column 是 integer；markers 必须配对且不能嵌套。标记 integer column 如果完全没有显式 bound card，默认 $[0, 1]$；一旦存在任何显式 card，未定义 sides 回到一般 $[0, +\infty)$ defaults。故单独 LI 表示 general integer，不再有隐式 upper 1；标记 integer 的 PL 也显式移除 upper。

bound card	精确含义
LO / UP	设置有限 lower / upper；可以为有理数。
FX	两侧设为同一个有限有理数。
FR / MI / PL	两侧 infinite / lower infinite / upper infinite；不允许 numeric field。

bound card	精确含义
LI / UI	设置 lower / upper 并声明 integer；该 card 的值必须是整数。
BV	integer 的 $[0,1]$ 与所有显式 bounds 相交，card 顺序无关；可选数值必须恰好为 1。

同一 side 重复定义必须相同；冲突的覆盖拒绝。相反 sides 的矛盾保留。若显式 upper 为负且没有显式 lower card，MPS 默认 lower 变为 $-\infty$；upper=0 则仍保留默认 lower=0。显式 lower 永远优先于此默认规则。BV 则始终和 $[0,1]$ 相交：BV+LO 2 得 $[2,1]$，BV+FR 得 $[0,1]$，BV+UP -1 得 $[0,-1]$，后者不会被负 upper 默认规则变为可行区间。

row RHS 为 b、range 为 r 时：

row 类型	canonical finite sides
L	$b-\lvert r\rvert \le a^\top x \le b$。
G	$b \le a^\top x \le b+\lvert r\rvert$。
E，$r \ge 0$	$b \le a^\top x \le b+r$。
E，$r < 0$	$b+r \le a^\top x \le b$。

没有 range 时保留原 equality 或单侧 constraint；range=0 生成 equality。

MPS 明确拒绝多个 N row、多个 vectors、初始 blank-name vectors、未知 section/bound type、SOS、quadratic/conic/indicator、semicontinuous/semiinteger 及其他未定义扩展。拒绝表明本 importer 没有接受这种方言，不能据此判断模型间关系。

5.3 LP v1 的自足文本规则

5.3.1 sections、名称与 coordinate order

`canonical` 依据输入文件扩展名选择 parser：大小写不敏感的 `.lp` 调用 LP importer，其他扩展调用 MPS importer；没有内容 sniffing。LP 输入必须 ASCII；反斜线开始到该物理行末的注释。除 line endings、tab 外的 control characters 拒绝。keywords 不区分大小写，每个 section header 独占一行。

必须先 objective，后 constraints，最后 End。Bounds 可选且位于 constraints 之后、type sections 之前。Binary、General、Semi 可按任意顺序放在 constraints/Bounds 之后，各至多一次；Bounds 不能出现在 type section 之后。

section	接受的 aliases
min objective	`Minimize, Minimum, Min`
max objective	`Maximize, Maximum, Max`
constraints	`Subject To, Such That, St, St., S.t.`
bounds	`Bounds, Bound`
binary	`Binary, Binaries, Bin`
general integer	`General, Generals, Gen`
empty semi marker	`Semi, Semis, Semi-Continuous`
end	`End`

Semi 只允许空 section；任何成员拒绝。End 后只能有空行或注释。

名称以 ASCII letter 或 underscore 开始；后续可用字母、数字及 `_.$#(),;!?@{}~%&`。名称区分大小写、最长 1,024 B、不截断。以 e/E 为首，后面缺省或紧接数字、e/E 的 exponent-ambiguous name 拒绝。名称不能是 section/type keyword、free、inf、infinity 或保留的未支持扩展 keyword：sos、lazy、scenario/scenarios、indicators、pwl、multi-objectives。quoted、square-bracket names 和数字/标点起首名称不支持；名称内括号是字面字符，不是表达式 grouping。

LP 没有 problem-name field，模型名固定为 `LP`。可选 objective label 只检查语法，不另存为模型字段。variable 顺序按 objective、rows、bounds、type lists 的首次文本出现决定；constraint 顺序按输入决定。constraint labels 保留且不能重复；unnamed rows 自动命名为 `_lp_row_0`、`_lp_row_1` 等。bounds/type-only 变量也保留。

5.3.2 表达式、续行与关系

表达式由 signed terms 组成：数值常数，或可选系数后接变量。首项以后每项都要 `+` 或 `-`。系数与变量必须有空白，例如 `2 x` 合法、`2x` 拒绝；operator 自身无需两侧空白，`x-y` 合法。符号可以与 magnitude 分离；objective 和 named row 的 coefficient/variable tokens 可跨物理行。不能使用乘号、power、quadratic brackets、grouping 或函数调用。零 objective 必须写 `0`，不能省略整个表达式。

若第一条 constraint 带 label，只有新 data line 开头的 `label:` 开始新 row，其他行继续当前 row。若第一条 constraint 不带 label，每个物理 constraint line 必须包含一条完整 unnamed relation；后续 named row 拒绝。不能由 parser 猜测未标号行是续行还是新 row。

单关系为 `affine <= affine`、`affine >= affine` 或 `affine = affine`。reader 减去右端，再把差的 constant 移到 bound。例如 $x + 1 \le y + 3$ 变为 $x - y \le 2$。range row 允许 `constant <= affine <= constant` 或反向的两个 `>=`；方向必须一致，两 endpoints 必须精确化简成常数。例如 $1 \le x + 2 \le 3$ 变为 $-1 \le x \le 1$；所有导入的 row Expr constant 为零。named range row 可跨行；strict inequalities 拒绝。

5.3.3 bounds 与 integrality

continuous/general integer 默认 $[0, +\infty)$；显式 bound 替换指定 side。LP 的负 upper 不会移除默认 lower zero，故只有 `x <= -1` 时为 $[0, -1]$。这与前节 MPS convention 不同，文件扩展选择是模型语义的一部分。

每个 Bounds record 占一物理行，只约束一个未缩放 variable。

```
0 <= x <= 3
3 >= x >= -2
x <= 4
-2 <= x
x >= -2
4 >= x
x = 1
1 = x
x free
-inf <= x <= +infinity
```

inf/infinity 不区分大小写，仅允许负 infinite lower 或正 infinite upper；infinite equality、正 infinite lower、负 infinite upper 均拒绝。同一 side 的重复 bound 必须相同；冲突 override 拒绝。Binary 是 integer 的 $\{0, 1\}$ 与 explicit bounds 的交集，例如 binary x 加 $2 \le x \le 7$ 为 $[2, 1]$，binary x free 为 $[0, 1]$。General 仅声明整数性，不隐含 upper=1。type lists 为以空白分隔的名称；重复或冲突 type declarations 拒绝。

LP 拒绝未知 sections/tokens、SOS、indicators、scenarios、quadratic/nonlinear syntax、非空 Semi、缺失必要 section/End、End 后 noncomment content 和未定义 dialect ambiguities。有理 fraction、双侧 affine relations 是本 importer 明确提供的扩展；vendor solver 接受范围不自动成为本 kernel 的输入语义。

5.4 精确数值与各层资源边界

MPS、LP 和 RPA 共用 `parse_q`。它接受带符号 integers、integer fractions、有限 decimal（含 `.1`、`1.`）及 e/E/d/D scientific notation，包括 `1/-2`；分母零拒绝。text 可以是未约分或负分母拼写，解析后规范化；binary Q 则必须已经规范，不能自动约分。NaN、infinity coefficients、hexadecimal floats、数字内部 whitespace 和 digit separators 不接受。无限 bounds 仅由上述 importer grammar 表示。

层与资源	当前上限
façade/CLI 每个 owned input file	512 MiB
RPA / Request text	256 MiB；JSON nesting capacity 128
MPS / LP text	512 MiB；每物理行 32,768 B
LP lexical tokens	20,000,000
model names / variables / rows	1,024 B / 2,000,000 / 2,000,000
MPS ROWS cards（含 objective）	2,000,000
coefficient entries / final model terms	20,000,000
decimal mantissa 或 fraction component digits	每部分 4,096
decimal exponent digits / absolute exponent	6 / 4,096
effective power of ten	绝对值至多 4,096
scalar text bytes	$2 \times 4,096 + 32$
exact intermediate numerator / denominator	各 65,536 bits
binary codec 文件 / 单 array / 累计 counted items	1 GiB / 10,000,000 / 20,000,000
binary magnitude / name preflight	每 magnitude 1,700 B / 1 MiB
binary proof depth	64，root=1
默认 verification work / live facts / depth	10^9 / 10^6 / 64

限制叠加适用，不能用宽松的一层覆盖严格的一层。例如 text parser 能构造的 65,536-bit Q 可能超过 MCM 的 1,700-byte magnitude 编码预算。`verify_bytes` 不适用路径读取的 512 MiB cap，但仍适用 codec、Request 和验证预算。theorem constructors 还有自己的维数和工作限制；生成模型也必须满足同一个 Model validation。预算耗尽没有浮点 fallback、wraparound 或“尽力接受”的模式。重要 allocations 使用 `try_reserve`；本接口不承诺第三方任意精度库的所有内部 allocation failures 或宿主强制终止都能恢复成 JSON。

work 在各方向、lift、injectivity、point refutation 与 improvements 间累积，嵌套 scopes 不重置。live facts 限制当前可见事实；proof nesting 与 JSON parser nesting 是两种不同限制。CLI 可以设 work 和 live facts，不能设置 depth。版本最低值与资源限制也不同：V10 本身不提高默认预算。

5.5 真实 CLI 与 Rust façade

当前命令如下；二进制名为 `rpx`，仓库构建命令为 `cargo build --release -p kernel-core --bin rpx`。

```
rpx canonical input.mps output.mcm
rpx canonical input.lp output.mcm
rpx assemble input.rpa output.rpx
rpx disassemble input.rpx output.rpa
rpx inspect-model model.mcm
rpx export-model model.mcm output.json
rpx verify --work 1000000000 --live-facts 1000000 \
  source.mcm target.mcm proof.rpx request.json
```

verify 的两个 options 可出现在 subcommand 后任何位置；paths 仍须恰好四个，顺序为 source、target、proof、request。数值分别解析为 u64 和 usize。其他 command 不接受这些 flags，没有 `--profile`、`--direction`、hash 参数或 certificate 自定 budget。未知 option、缺参数、路径数量错误返回 usage INVALID。

façade 的路径函数与 CLI 同名操作对应：`canonicalize_path`、`assemble_paths`、`disassemble_paths`、`inspect_model_path`、`export_model_paths`、`verify_paths`。需要内存集成时使用 `api::verify_bytes(source,target,proof,request,limits)`，前三项为 byte slices、request 为 text；不触及文件系统。入口返回 `Result<serde_json::Value, api::Error>`。更底层的 `verify::verify_request` 接受内存中的 Model/Certificate/Request 与 Limits，并自行再次验证两个完整模型；带位置的接口为 `verify_request_with_diagnostics`。legacy `verify` 接受 Claim slice，只请求方向，不接受 points/bijections/improvements。独立 LP contradiction 和完整 injectivity 库入口分别为 `verify_lp_refutation`、`verify_injectivity`，并有相应 diagnostics 版本。低层 `verify_injectivity` 只检查给定完整 map 的单射性；它不独立建立总可行性、与证书索引的绑定或完整双射。完整合约验收使用 `verify_request`。

路径读取先检查 metadata length，再用限长 read 确保文件增长也不超过 cap；实际用于验证的是取得的 owned bytes。这不是操作系统级多文件事务：如需 source/target/proof/request 的一致业务快照，由调用者先获取并通过 `verify_bytes` 绑定。output 在内容全部生成后创建父目录并写入指定文件；路径函数可能替换现有 output，不承诺 atomic rename 或自动 backup。

5.6 报告、诊断与模型 JSON

每次 CLI invocation 向 stdout 打印一个 JSON value。verify 成功包括 status、claims/points/bijections counts、statistics、limits、`model_decode_us`、`proof_decode_us`、`execute_us`、`total_us`；路径调用另有 `read_us`，内存调用没有该字段。非空 improvement 请求增加 `improvements` count；新增 V10 statistics 零值省略。这些 timings 是本次调用诊断量，不能单独解释为隔离 benchmark。

failure 必含 status、message、code；已知时增加 location 与 limits。

```
{"status":"INVALID",
 "message":"INVALID obligation mismatch: expected Eq; obligation 3",
 "code":"obligation_mismatch",
 "location":{"claim":0,"block":0,"obligation":3},
 "limits":{"work":1000000000,"live_facts":1000000,"depth":64}}
```

code 是错误词汇生成的 snake case 诊断辅助，不是冻结枚举；稳定分类应以 status 为准。Location 全部 indices 为零基：claim/point 是 Request 各自数组下标，block 是顶层 certificate Block 下标，step 是最内层活动 Block/Derivation 的 Step 下标，obligation 是当前 terminal 的实际检查次序下标。没有相应语

义的位置省略字段。improvement、fiber refutation 等不是顶层方向 Block，不能从不存在的 claim/block 字段推断坐标。diagnostics 只是 best-effort error attribution，不参与数学决定、统计或预算。CLI 在有效 verify 参数解析后的失败上附加 effective limits；库调用者可用 `Error::with_limits` 附加。

`inspect-model` 报告模型名、variables/rows、integer count 和 row/目标总 nonzeros；`export-model` 通过 `model::to_json` 输出 name、variables、rows、objective。每个 rational 为 canonical scalar string，bounds 缺侧为 null，terms 是有序的 `[variable,coefficient]` pairs。JSON object key 顺序不具有语义，数组次序保留模型坐标。这是 export schema，不能直接使用 Model 的普通 Serialize derive 作为同样的 scalar interchange，也没有当前 CLI 的 JSON-model import command。

5.7 信任结论与外部证据

接受边界包含精确数值 parser/算术、MPS/LP importers、Model validation、MCM/RPX codec、Request matching、scalar/scope checker、完整 obligation construction、point 与 injectivity checker，以及各静态 lift/theorem 的数学 lemma 和 Rust 实现。扩展的 old-scope premise checking 只证明其前提；生成模型合法且 child proof 通过，仍不能独立证明联合存在性 lemma。这一项必须明确列入 TCB。

certificate generator、求解器、Python oracle、model-specific structure scripts、外部 SHA receipts、benchmark 和 corpus replay 都不向 kernel 提供 proof facts。独立测试、字节攻击、构造 oracle 与 source review 是实现可靠性的证据，不是 Rust 实现的形式验证。V9/V10 的工作 case 成功也不冻结草案。验证已准许请求不自动获得未请求的双向等价、最优性、可行性或 global inclusion。

本章以 `api.rs`、`main.rs`、`model.rs`、`lp.rs`、`verify.rs` 为实现依据；输入方言详解对应 `docs/MPS_V1.md`、`docs/LP_V1.md`，接口说明对应 `docs/API.md`。

第六章　RPA 文本汇编与工具链

RPA 是 RPX certificate 的 strict JSON 文本表示。当前 assembler 解析结构和精确 scalars、核对最低 feature version 并调用 binary encoder；它不执行 proof search，不生成缺失 obligations，也不拥有独立于 verifier 的证明权力。成功的 `ASSEMBLED` 只说明证书可编码。本章给出可直接编写生成器的 schema；二进制逐字段布局见第四章。

6.1　精确 header 与 JSON 词法

文件采用 UTF-8，必须以精确 ASCII bytes `RPA N` 后接一个 LF 开始，其中 N 为十进制的 1、2、3、4、5、6、7、8、9、10 之一。header 前不能有 BOM、空行或 whitespace；N 不能有前导零，header 不能使用 CRLF、额外空格、注释或 minor version。header 后的 body 是一个完整 JSON Certificate object；JSON 自身允许正常空白与换行，包括 CRLF。body 后可以有 JSON whitespace，不能有第二个顶层值。

body 不接受 comments、trailing commas、unknown fields、unknown enum variants 或重复已声明 record fields。名称和 variant 拼写区分大小写。没有旧 D0 的 `.source`、`.contract`、`.expr`、`%fact`、mnemonic instructions、symbol labels、includes、macro、ScalarPool 或 ExprId。

所有数学数值写成 JSON string，例如 `"1/3"`，不能写成 JSON number 0.3333。coordinate IDs、Local IDs、claim indices、node IDs、bits、distance 和 maximum index 都是无符号 JSON integers，在 u32 范围；Plotkin label 是 u64。不能加引号，不能用负数或浮点形式表示 ID。boolean 必须为 `true` 或 `false`，不是 0/1 字节。

```
RPA 10
{"claims":[],"blocks":[],"points":[],"bijections":[],
 "improvements":[]}
```

这是可解析的空 certificate 示例，不是可通过空 Request 的数学证明。RPA header 只给出允许的能力上限；实际使用功能不能超过 N。例如 RPA 1 body 内出现 Cases，即使该 branch 不可行也拒绝；RPA 10 body 只有 V1 records，装配时会输出 RPX 1。V1–V8 是冻结协议，V9/V10 是工作草案；写 header 10 不改变其冻结状态。

6.2　精确 scalar 字符串

字符串形式	例子与数学解释
signed integer	`"17"`、`"-3"`、`"+0003"`。
integer fraction	`"4/7"`、`"-22/13"`、`"1/-2"`；分子分母为 signed integers，分母不得为零。
finite decimal	`"0.125"`、`".125"`、`"1."`，用整数十进制换算。
scientific decimal	`"1e-6"`、`"-2.5E+3"`、`"2D+3"`，接受 e/E/d/D。

拼接 mantissa 数字为整数 d、小数位数为 k、指数为 t、符号为 s，值为 $s\,d\,10^{t-k}$。所有步骤精确；例如 `"0.125"` 得 1/8，`"1/-2"` 得 $-1/2$，`"0/7"`、`"-0"` 得零。text parser 接受未约分 fraction 并规范化；

binary decoder 独立拒绝未规范 wire Q，不能因证书来自 RPA 就省略该检查。

NaN、Inf/infinity、hex floats、循环小数、digit separators 和 scalar 内 whitespace 拒绝。资源预算限制文本长度、digits、exponent 与精确 arithmetic 位数，见第五章。反汇编只输出 canonical integers 或 coprime `p/q`，分母为正；整数无 `/1`，正数无加号，零为 `"0"`。原 decimal、exponent、前导零和 negative-denominator 拼写不会保留。

6.3 顶层 Certificate 与独立 Request

下表使用 schema 记法：`T[]` 是完整 T values 的 JSON array；`[A,B][]` 是 pair arrays，不能替换为 object。所有实际 JSON keys 都带双引号；表中的省略 default 表示该 field 可以缺席，并不允许任意遗漏 required fields。

record	key	value 类型	缺省
Certificate	claims	Claim[]	必需
	blocks	Block[]	必需
	points	PointWitness[]	[]
	bijections	BijectionWitness[]	[]
	improvements	ImprovementWitness[]	[]
Request	claims	Claim[]	必需
	points	PointClaim[]	[]
	bijections	BijectionClaim[]	[]
	improvements	ImprovementClaim[]	[]
Expr	constant	scalar string	必需
	terms	[u32, scalar string][]	必需
Block	steps / end	Step[] / End	必需
Derivation	steps / end	Step[] / DerivationEnd	必需
PointWitness	claim / values	PointClaim / scalar string[]	必需
	refutation	Block 或 null	null
`BijectionClaim`	`forward_claim / reverse_claim`	u32 / u32	必需
`Bijection Witness`	claim / injectivity	BijectionClaim / EqualityProof	必需
`EqualityProof`	steps / checks	Step[] / Ref[]	必需
`Improvement Claim`	target / integers / positive	bool / bool / Expr	必需
`Improvement Witness`	claim / block	ImprovementClaim / Block	必需

Request 没有 RPA header，是独立普通 JSON。它也可以是 legacy Claim array，此时 points、bijections、improvements 都为空：

```
[{"reverse":false,"input_integers":false,
  "output_integers":false,"objective":"Equal",
  "projection":[],"inverse":null}]
```

object Request 的四个 arrays 按各自顺序匹配 certificate；point/bijection/improvement 只匹配其 claim，没有 proof witness。certificate claims 和 blocks 数量必须一致。空 claims 可以用于 point-only 或 improvement-only Request，但 claims、points、improvements 全为空的 Request 拒绝。bijections 本身不能替代必要的 directional claims。调用者应从自己意图构造 Request，不能在读取 certificate 后盲目复制它并声称完成独立授权。

6.3.1 Claim 的所有 keys

key	value 类型与含义	缺省
reverse	bool；false 为 source 到 target，true 为 target 到 source。	必需
`input_integers`	bool；是否采用 actual input 的 integer predicates。	必需
`output_integers`	bool；是否要求 actual output 的 integer predicates。	必需
objective	`"None"`、`"Equal"` 或 `"NonIncreasing"`。	必需
projection	[u32 input, u32 output][]；原始坐标观察相等。	必需
inverse	opposing Claim index 或 null。	null
`input_objective_lower`	scalar string 或 null；V3 输入完整目标下界。	null
`affine_projection`	[Expr input, Expr output][]；V5 原始 affine observations。	[]
`input_objective_level`	scalar string 或 null；V9 objective-level 条件输入域。	null
`input_halfspace`	Expr 或 null；V10 $H(x) \le 0$ 条件输入域。	null

没有字符串 profile 或总体 optimum-equivalence flag；需要的关系由独立 ordered Claims 表达。input/output integer modes 可以不同，每个方向必须按实际 Request 执行。present objective level 不是 lower bound、attainment 或 optimum 的声明；present halfspace 不是证明该表达式在全模型上有效。有限 lower request 在 restricted domain 上是条件结论；global bound 必须单独请求。

6.4 Expr、Ref 与显式事实 IDs

一个 affine expression 用 `constant` 加有序非零 coefficients：

```
{"constant":"-1/2","terms":[[0,"2"],[3,"-5/7"]]}
```

terms 的 IDs 严格递增、没有 duplicate、不含精确零。assembler 不负责 symbolic expansion，也不自动 sort、merge 或删除 Expr 的零项；生成器必须先构造规范表达式。文本中系数约分后若为零仍非法。Lin terms 是另一种 pair array，其第一项为 Ref，不受 Expr 的 coordinate-order 规则代替。

Ref 的完整文本形式：

```
{"VarLower":0}  {"VarUpper":0}  {"VarInt":0}
{"RowLower":2}  {"RowUpper":2}  {"RowEq":2}
{"Local":17}    {"Assumption":0}
"Exact"
```

每个非 Exact Ref object 恰好一个 variant key。静态 Ref 必须对应实际有限 side、equality 或可用 input integer；不是仅因 ID 可写而拥有事实。`"Exact"` 是 unit variant string，不是 `["Exact"]`、`{"Exact":0}` 或可自由借用的事实。

Local ID 是显式 u32，作者可以选 17 为第一条 producer ID；不会被 assembler 重排为 0。definition 必须在使用前通过检查，不能覆盖 live ID；Drop 后可以在当前 scope 重用。child 删除/重定义不泄漏到 parent 或 sibling。Assumption 使用活动假设的外层先、内层后数组顺序；最外层为 0，不能套用旧 `@case[0]` 的“最近一层”含义。各方向、point refutation、injectivity proof、improvement Block 从各自 fresh scope 开始。

6.5 Step 与局部 Derivation 的 JSON variants

Enum 使用 externally tagged form，record payload 写为 `{"Variant":{fields}}`。Kind 的值恰为 `"Le"`、`"Eq"`、`"Int"`，不能全大写或写 numeric tags。

variant	payload keys 和类型	最低版
Lin	`id:u32, kind:Kind, constant:scalar, terms:[Ref,scalar][]`	1
Round	`id:u32, le:Ref, integer:Ref`	1
Drop	`ids:u32[]`	1
EqPair	`id:u32, positive:Ref, negative:Ref`	2
Cases	`id:u32, kind:Kind, expression:Expr, split:Expr, integer:Ref|null, left:Derivation, right:Derivation`	2
Plotkin	`id:u32, bits:u32, distance:u32, variables:[u32,u64][], cliques:Ref[]`	3

这些 listed keys 除 Cases 的 integer 可省略为 null 外都必需。Round 的 integer 为一个 Ref，不能省略或写 null。Lin 没有 out Expr：checker 根据已检查 facts 精确计算输出 expression。Cases 的 expression 是要输出的同一 parent proposition，split 是两支划分表达式；左右 Derivation 必须证明该 expression/kind 或 contradiction。

```
{"Lin":{"id":17,"kind":"Le","constant":"-1/3",
        "terms":[[{"VarUpper":0},"2/3"]]}}
{"EqPair":{"id":18,"positive":{"Local":17},
           "negative":{"RowLower":4}}}
{"Drop":{"ids":[17,18]}}
```

以上是独立语法片段，不声称组合成有效数学证明。

Derivation 的 end 为下列二者之一：

```
{"Witness":{"Local":17}}
{"Absurd":{"Local":18}}
```

这里直接 payload 是 Ref，没有 `fact` key。DerivationEnd 不含 map/checks；它不是 complete Return，也不能用 lift 或 improvement endpoints。Cases 的左右 end 不要和下一节的 Block Absurd 混用。

6.6 Block End 与 witness records

End variant	payload keys 和类型	最低版
Return	`map:Expr[], checks:Ref[]`	1
Split	`expression:Expr, integer:Ref|null, left:Block, right:Block`	1
Absurd	`fact:Ref`	1
FlowLift	`witness:FlowWitness, body:Block`	4
IntervalLift	`intervals:IntervalWitness[], body:Block`	5
MenuLift	`menus:MenuWitness[], body:Block`	7
SignedLift	`decompositions:SignedWitness[], body:Block`	8
SubflowLift	`witness:SubflowWitness, body:Block`	10
ImproveReturn	`map:Expr[], checks:Ref[], margin:ImprovementMargin`	10
`Positive Refutation`	`fact:Ref`	10

Split.integer 可省略为 null，其他 keys 必需。Block Absurd 必须写 `{"Absurd":{"fact":Ref}}`，它与 DerivationEnd 的 `{"Absurd":Ref}` 形状不同。

Return.map 为 output 所有坐标的完整 tuple；checks 按 kernel 规定顺序覆盖 bounds、rows、integer requirements 和全部请求项。assembler 不加载实际模型，所以即使遗漏 checks 可编码，最终 verify 仍拒绝。每个 Split child 有自己的 complete terminal；不能用空分支或 parent 的 partial map 省略 obligations。所有 lift witness entries 的前提先在旧 scope 一起核对，之后 body 才使用生成的坐标/rows。

所有 witness 是普通 record objects，不带同名 enum wrapper：

witness record	required keys 与类型
FlowWitness	`nodes:u32, arcs:[u32,u32][], groups:FlowGroup[]`
FlowGroup	`source:u32, commodities:[u32,scalar][], flows:Expr[], nonnegative:Ref[], balances:Ref[]`
IntervalWitness	`lower:Expr[], upper:Expr[], checks:Ref[]`
MenuWitness	`maximum_index:u32, activation:Expr, index:Expr, checks:Ref[]`
SignedWitness	`capacity:scalar, flow:Expr, activation:Expr, checks:Ref[]`
SubflowWitness	`nodes:u32, arcs:[u32,u32][], groups:SubflowGroup[]`
SubflowGroup	`source:u32, demands:[u32,scalar][], flows:Expr[], nonnegative:Ref[], balances:Ref[]`

空数组仍显式写为 []；这里没有 omitted defaults。Flow 使用 commodities，Subflow 使用 demands，不能互换 key。arcs 的 pair 是 tail/head；commodities/demands 的 pair 是 sink/demand。维数、premise order、joint extension row/variable order 和数学规则见第三章。

ImprovementMargin 是 scalar newtype enum：

```
{"Constant":"1/2"}
{"Predicate":"1"}
```

q 必须严格正；Predicate(q) 表示 $qP(x)$。普通方向、fiber-refutation 和 Cases 不能借用 strict improvement root context；正文 End table 的最低版是编码能力要求，不是任意 context 的执行许可。

6.7 Point、Bijection 与 Improvement contracts

PointClaim 共有四个 externally tagged record variants：

variant	payload keys 和类型	最低版
Feasible	`target:bool, integers:bool, objective:scalar|null`	3
Strictness	`inclusion_claim:u32`	3
`FiberStrictness`	`inclusion_claim:u32`	6
ImageExclusion	`point_in_target:bool, map:Expr[]`	10

Feasible.objective 可省略为 null；其他 keys 必需。PointWitness.values 为所选 actual model 所有坐标的 scalar strings。FiberStrictness 必须提供 refutation Block；其余三个 variants 必须 refutation=null 或缺席。提供格式合法但不需要的 refutation 也会被 verifier 拒绝。

```
{"claim":{"Feasible":{"target":false,"integers":true,
                      "objective":"0"}},
 "values":["0","1/2"],"refutation":null}
{"claim":{"ImageExclusion":{"point_in_target":false,
   "map":[{"constant":"0","terms":[[0,"1"]]}]}},
 "values":["2"],"refutation":null}
```

这是两种 witness 的语法示意，是否可行取决于实际完整模型。Strictness 和 FiberStrictness 的 inclusion index 要指向已请求、实际通过且未 restricted 的 LP inclusion；不能把任选 point 宣称为 strictness。ImageExclusion 由调用者绑定完整 affine map，所选 point 及其 image 都按 actual LP 检查，不声明整体包含或 partial-map 的所有 completions 都被排除。

BijectionClaim 为 `forward_claim` 与 `reverse_claim` 两个 u32 keys。BijectionWitness 的 `claim` 是该 record，`injectivity` 是仅含 steps/checks 的 EqualityProof。它没有 end，没有内嵌 `"Bijection"` variant，也没有选择少量 observations 的 shortcut。两 opposing Claims 的 directions/modes 必须兼容，forward 要有 root affine Return；reverse 每个 Return 及独立成对完整输入域 injectivity 都须通过。V9/V10 restricted domains 不适用于既有 V7 bijection contract。

ImprovementClaim 恰为 target、integers、positive 三个 keys；ImprovementWitness 恰为 claim、block。

```
{"claim":{"target":false,"integers":false,
          "positive":{"constant":"0","terms":[[0,"1"]]}},
 "block":{"steps":[],
          "end":{"PositiveRefutation":{"fact":{"RowUpper":0}}}}}
```

它请求 $P(x) > 0$ 的每个实际输入存在同一实际模型中的严格改善点；不是源到目标映射。这个示意 endpoint 是否成立取决于实际 row 是否精确证明 $P \leq 0$。完整 witness 必须使用 ImproveReturn、PositiveRefutation 或适当的 Absurd/Split/lifts；普通 Return 在 improvement context 拒绝。

6.8 一套可直接执行的最小程序

取一个连续变量 $x \in [0,1]$、无 rows、目标 x 的模型。将下列内容保存为 `identity.lp`：

```
Minimize
  obj: x
Subject To
Bounds
  0 <= x <= 1
End
```

请求一条 source 到 target 的 objective equality，source/target 选同一个实际 MCM。独立 `request.json` 使用前文的 Claim array。保存下列完整 certificate 为 `identity.rpa`：

```
RPA 1
{"claims":[{"reverse":false,"input_integers":false,
 "output_integers":false,"objective":"Equal",
 "projection":[],"inverse":null}],
 "blocks":[{"steps":[],"end":{"Return":{
 "map":[{"constant":"0","terms":[[0,"1"]]}],
 "checks":[{"VarLower":0},{"VarUpper":0},"Exact"]
 }}}]}
```

同一 map $y = x$ 生成三个 obligations：$-x \le 0$、$x - 1 \le 0$、$x - x = 0$，分别由两个 actual bounds 和 Exact 证明。没有 row 或 integrality obligation；projection=[] 也不生成观察 equality。

```
rpx canonical identity.lp identity.mcm
rpx assemble identity.rpa identity.rpx
rpx verify identity.mcm identity.mcm identity.rpx request.json
rpx disassemble identity.rpx recovered.rpa
```

最后 verify 为 `VERIFIED`；前两个命令成功只代表导入、装配。这一条方向不会自动请求 reverse、inverse 或 bijection。需要它们时应在 Request 与 certificate 中同时增加相应完整数据。

6.9 规范反汇编、失败与实现依据

反汇编读取 complete RPX certificate 后调用 `certificate_version`，输出最低所需 `RPA N` header 和 pretty JSON，末尾带 LF。scalar strings 规范化；空 optional certificate arrays、空 `affine_projection`、缺席 optional scalar fields 和缺席 point refutation 可能省略，inverse 则由当前 serializer 输出为 null。JSON key 次序不能用来确定数学身份；array 次序、显式 IDs、每个 field 的实际值保留。不存在源行、comment、标签或原 scalar 拼写的 round-trip 保证。高于最低版的 binary 反汇编再装配可能降版，详见第四章。

RPA parser 本身至多接受 256 MiB 和 serde JSON recursion capacity 128；proof version discovery/encoding 另检查 depth、count、file、Q 与 canonical Expr budgets。JSON 的 object/array wrappers 也占 nesting，一个尚未达到 kernel proof depth=64 的文本可能先耗尽 JSON capacity。该情况返回 `RESOURCE_LIMIT`，不是 INVALID mathematics。header/syntax/unknown fields/variants 等错误为 `INVALID`。某些 proof 结构在 text parsing 阶段可构造，后续 encoder 或 verifier 才拒绝；生成器不能把

任意一个较早阶段的成功当成 acceptance。

真实工具链只有第五章列出的 commands。当前没有 symbol macro compiler、automatic obligation filler、通用 theorem registry、host plugin opcode 或 solver certificate conversion command。生成器、Python construction scripts 与 coordinate index files 都处于不可信准备层；它们可以帮助构造上述明确数据，最后仍须由 complete actual models、独立 Request 和 verifier 决定。

文本 grammar 的实现依据为 `crates/core/src/rpa.rs`、`proof.rs` 与 `model.rs` 的 serde definitions；wire feature minimum 来自 `codec::certificate_version`。V10 的 image、improvement、subflow 和 halfspace 均包含在上述 schema，不应只依据旧版 `docs/RPA_V1.md` 推导后续字段。

第七章　完整证书示例与验收案例

本章的前三份证书都是当前 CLI 可执行的完整双向程序。每份包含两份完整 LP 模型、一份严格 JSON RPA 和调用者独立提供的 request；正文直接引用仓库文件，没有省略的变量、模型行、证据或摘要占位值。它们采用已冻结的 RPA/RPX 1。MCM 和 RPX 由现有命令生成到临时目录，证书不含授权接受的模型 hash。

7.1　复核入口与结果

从仓库根目录执行：

Listing 7.1: 重建、回读与负例复核

```
cargo build -p kernel-core --release --locked
python3 tools/check_rpx_manual.py --kernel target/release/rpx --out docs/.build/manual-examples.
    json
```

脚本对每份示例重新 canonicalize 两份 LP、assemble、verify，再 disassemble、assemble 并要求 RPX 字节相同、再次 verify。它还独立运行三种逻辑变异和一次零 work 预算检查。2026 年 10 月 13 日当前源码重建后的复核全部通过；完整 receipt 在 `docs/rpx/examples/verification.json`，记录实际可执行文件与输入的 SHA-256、计数和错误报告。SHA-256 仅用于复核产物身份。

示例	claims	RPX bytes	obligations	branches	work
冗余行删除	2	300	13	0	56
等式消元	2	343	14	0	73
分段绝对值	2	351	9	1	42

表中的 obligations 是实际执行的检查次数。分段示例的两个 Forward 叶子都被核验，故贡献六次检查，Reverse 再贡献三次。它不是 D0 的分支 DONE 位图大小。计数和时钟测量的区别见第八章。

7.2　示例 A：删除冗余上界行

7.2.1　完整模型与调用者要求

Source 为 $0 \le x \le 1$，有 $x \le 1$ 和 $x \le 2$ 两行；Target 为 $0 \le y \le 1$，只保留 $y \le 1$。目标都恒为零，变量均连续。LP 的首次出现顺序使唯一坐标编号为 0，Source 行编号为 cap=0、loose=1。

Listing 7.2: A：完整 Source LP

```
Minimize
 obj: 0 x
Subject To
 cap: x <= 1
 loose: x <= 2
Bounds
 0 <= x <= 1
End
```

Listing 7.3: A：完整 Target LP

```
Minimize
 obj: 0 y
Subject To
 cap: y <= 1
Bounds
 0 <= y <= 1
End
```

Listing 7.4: A：独立 caller request

```
[
 {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,0]],"inverse":1},
 {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,0]],"inverse":0}
]
```

request 明确要求两方向目标相等、坐标投影相等和互为 affine inverse。因此该例确立完整可行集的目标保持双射；不能从仅有双向可行性的证书推测出这些额外关系。

7.2.2 完整 RPA 与核验

Listing 7.5: A：完整 RPA 1 证书

```
RPA 1
{
 "claims":[
  {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,0]],"inverse":1},
  {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,0]],"inverse":0}
 ],
 "blocks":[
  {"steps":[],"end":{"Return":{
   "map":[{"constant":"0","terms":[[0,"1"]]}],
   "checks":[{"VarLower":0},{"VarUpper":0},{"RowUpper":0},"Exact","Exact","Exact"]
  }}},
  {"steps":[{"Lin":{"id":0,"kind":"Le","constant":"-1","terms":[[{"RowUpper":0},"1"]]}}],
   "end":{"Return":{
    "map":[{"constant":"0","terms":[[0,"1"]]}],
    "checks":[{"VarLower":0},{"VarUpper":0},{"RowUpper":0},{"Local":0},"Exact","Exact","Exact"]
   }}}
 ]
}
```

Forward 和 Reverse 都取恒等 map。Reverse 的 Lin 定义

$$\mathrm{LE}\big((y-1)-1\big) = \mathrm{LE}(y-2),$$

所以 Source 的 loose 行也得到精确匹配。每个 Return 按 kernel 顺序列出 bounds、rows、objective、projection 和 inverse 证据；Forward 有六个，Reverse 有七个。没有整数性义务，也没有隐含不存在的行下界。

单独运行本例的真实命令为：

Listing 7.6: A：从 LP 到 VERIFIED

```
mkdir -p docs/.build/manual-redundant
target/release/rpx canonical docs/rpx/examples/redundant/source.lp docs/.build/manual-redundant/
    source.mcm
target/release/rpx canonical docs/rpx/examples/redundant/target.lp docs/.build/manual-redundant/
    target.mcm
target/release/rpx assemble docs/rpx/examples/redundant/proof.rpa docs/.build/manual-redundant/
    proof.rpx
target/release/rpx verify docs/.build/manual-redundant/source.mcm docs/.build/manual-redundant/
    target.mcm docs/.build/manual-redundant/proof.rpx docs/rpx/examples/redundant/request.json
```

最后一条返回 `VERIFIED`，退出码 0；前面的成功状态分别为 `CANONICALIZED` 和 `ASSEMBLED`。

7.3 示例 B：利用等式消去一个变量

7.3.1 完整模型与调用者要求

Source 的 $x_0, x_1 \in [0,1]$ 满足 $x_0 + x_1 = 1$，目标 $f_S = x_0$；Target 的 $y \in [0,1]$ 无行，目标 $f_T = 1 - y$。保留 x_1，重建 $x_0 = 1 - y$。目标常数 1 必须参与 canonicalization 和证明。

Listing 7.7: B：完整 Source LP

```
Minimize
 obj: x0 + 0 x1
Subject To
 balance: x0 + x1 = 1
Bounds
 0 <= x0 <= 1
 0 <= x1 <= 1
End
```

Listing 7.8: B：完整 Target LP

```
Minimize
 obj: 1 - y
Subject To
Bounds
 0 <= y <= 1
End
```

Listing 7.9: B：独立 caller request


```
[
 {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[1,0]],"inverse":1},
 {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,1]],"inverse":0}
]
```


两个 projection 数组分别要求 $y = x_1$ 和 $x_1 = y$。两个 inverse 字段要求恢复全部输入坐标，不仅是被保留坐标；它们在不同维数的可行集上仍可成立。

7.3.2 完整 RPA 与核验

Listing 7.10: B：完整 RPA 1 证书


```
RPA 1
{
 "claims":[
  {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[1,0]],"inverse":1},
  {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[[0,1]],"inverse":0}
 ],
 "blocks":[
  {"steps":[{"Lin":{"id":0,"kind":"Eq","constant":"0","terms":[[{"RowEq":0},"-1"]]}}],
   "end":{"Return":{
    "map":[{"constant":"0","terms":[[1,"1"]]}],
    "checks":[{"VarLower":1},{"VarUpper":1},{"Local":0},"Exact",{"Local":0},"Exact"]
   }}},
  {"steps":[],"end":{"Return":{
   "map":[{"constant":"1","terms":[[0,"-1"]]},{"constant":"0","terms":[[0,"1"]]}],
   "checks":[{"VarUpper":0},{"VarLower":0},{"VarLower":0},{"VarUpper":0},"Exact","Exact","Exact","Exact"]
  }}}
 ]
}
```


Forward 的目标差与第一个 inverse 坐标差都是

$$(1 - x_1) - x_0 = -(x_0 + x_1 - 1).$$

`Local:0` 是由实际 input equality 的负倍数推出的 EQ，不能替换为 `Exact`。Forward 有六个 obligations：两个 bounds、目标、投影、两个 inverse 坐标。Reverse 的四个 bounds 归一化依次为 $y-1, -y, -y, y-1$，分别使用 Target 的 upper、lower、lower、upper；等式行、目标、投影和一个 inverse 坐标随后都精确恒为零，共八个 obligations。

复核脚本调用与示例 A 相同的 `canonical`、`assemble`、`verify` 命令，参数换为 `rpx/examples/elimination/` 的四份完整输入；独立 request 始终是该目录的 `request.json`。当前实际结果为两方向 `VERIFIED`、14 obligations。

7.4 示例 C：分段绝对值 mapping

7.4.1 完整模型与调用者要求

Source 的 $x \in [-1, 1]$，Target 的 $y \in [0, 1]$，都没有模型行，目标都为零。Forward 取 $y = |x|$，Reverse 取 $x = y$。request 只要求双向可行性和目标相等，不要求 projection 或 inverse：x 与 $-x$ 可能映到同一 y。

Listing 7.11: C：完整 Source LP

```
Minimize
 obj: 0 x
Subject To
Bounds
 -1 <= x <= 1
End
```

Listing 7.12: C：完整 Target LP

```
Minimize
 obj: 0 y
Subject To
Bounds
 0 <= y <= 1
End
```

Listing 7.13: C：独立 caller request

```
[
 {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[],"inverse":null},
 {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[],"inverse":null}
]
```

7.4.2 完整 RPA 与核验

Listing 7.14: C：完整 RPA 1 证书

```
RPA 1
{
 "claims":[
  {"reverse":false,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[],"inverse":null},
  {"reverse":true,"input_integers":false,"output_integers":false,"objective":"Equal","projection":[],"inverse":null}
 ],
 "blocks":[
  {"steps":[],"end":{"Split":{
   "expression":{"constant":"0","terms":[[0,"1"]]},"integer":null,
   "left":{"steps":[],"end":{"Return":{
    "map":[{"constant":"0","terms":[[0,"-1"]]}],
    "checks":[{"Assumption":0},{"VarLower":0},"Exact"]
   }}},
   "right":{"steps":[],"end":{"Return":{
    "map":[{"constant":"0","terms":[[0,"1"]]}],
    "checks":[{"Assumption":0},{"VarUpper":0},"Exact"]
   }}}
  }}},
  {"steps":[{"Lin":{"id":0,"kind":"Le","constant":"-1","terms":[[{"VarLower":0},"1"]]}}],
   "end":{"Return":{
    "map":[{"constant":"0","terms":[[0,"1"]]}],
    "checks":[{"Local":0},{"VarUpper":0},"Exact"]
   }}}
 ]
}
```

Forward 的 Split 两支分别假设 $x \le 0$ 和 $-x \le 0$，覆盖所有实数。left 的完整 Return 定义 $y = -x$，其 lower 是 assumption，upper 是 Source lower $-x-1 \le 0$。right 定义 $y = x$，lower 是 assumption，upper 是 Source upper。两支各自提供同一个完整三项检查清单；Split 后没有继续执行或汇合不完整 map 的代码。

零点同时满足两支条件，两支给出的值都为零。当前 verifier 核验两个完整子树；此证书与 CLI 不提供 D0 中建议的独立数值解重建器或 SEALED_PIECEWISE 状态。Reverse 用 Lin 从 $-y \le 0$ 推出 $-y-1 \le 0$，再直接证明上界和目标。当前结果为 VERIFIED、九个 obligations、一个 branch point。

7.5 每份示例的拒绝与资源检查

实际变异	当前结果	拒绝或中止原因
删掉 Forward Return 的最后一个 check；C 在 left 叶子删除	INVALID，missing_obligation	kernel 从完整模型和 request 生成的义务没有对应证据。
原证书不动，把 caller 的首个 objective 改为 None	INVALID，requested_contract_mismatch	caller request 必须完整相等，即使新 request 更弱也不能代替原要求。
改变 Reverse map 第一个表达式的常数；A/C 改为 1，B 改为 0	INVALID	A 的 Forward inverse 首先成为非恒真 Exact；B/C 的 bounds 证据表达式不匹配。
全部正例不动，使用 --work 0	RESOURCE_LIMIT，退出码 3	共享工作预算耗尽，没有建立证书成立或不成立。

三份示例的九次逻辑变异全部退出 2，三次预算检查全部退出 3。证明文法合法和 assemble 成功不会使错误的 map 或不完整证据成为 VERIFIED。脚本检查 process exit 与 JSON status 两者，不以日志中出现成功单词作为验收。

7.6 完整 corpus：严格改进与条件最优投影

小示例不替代完整模型的实际证书。cases/test36/ 的 RPA 10、两份 MCM 和独立 request 绑定真实 Source 的 37,816 个坐标与 37,617 行，以及真实 Target 的 37,616 个坐标与同样行数。令 P 为全部 200 个原始非负连续 terminal 的和。完整 request 同时要求：

1. 所有 Source MI 可行点有完整 Target MI 像，目标 NonIncreasing。
2. 所有 Target MI 可行点有完整 Source MI 恢复，目标 Equal，并保留全部 37,616 个 retained 坐标。
3. 在 caller 明确绑定的原始半空间 $P \le 0$ 内，Source 到 Target 的完整 map 目标 Equal，并保留上述全部坐标。
4. 每个满足 $P > 0$ 的 Source MI 点都有一个完整 Source MI 严格改进。

第四项迫使任何已达到的 Source minimizer 满足 $P = 0$，因为原始 bounds 保证 $P \ge 0$。第三项随后适用于这个 minimizer，建立它本人的 retained projection；第二项恢复 Target minimizer。仅有第四项不能证明 retained projection，另取一个更优点可能改变这些坐标。条件 claim 的 input_halfspace 与 improvement 的 positive 都精确绑定全部 200 个 original terminal；删项、反向或换 caller request 均不能代表同一结论。

条件 proof 在完整原始模型末尾追加 $P \le 0$，由原始 lower bounds 和 EqPair 得到 $P = 0$，再证明

每个 terminal 为零；它没有用外部裁剪模型替代输入。改进 proof 分别使用 predicate margin $4850P$ 和 constant margin 858。连续 terminal 不借用整数 guard；保存的 $P = 1/4850$ 可行控制点说明正 terminal 仍然可能 Source-feasible。

这确立最小值保持和所有最优点的 retained projection，不确立无条件可行集投影相等、LP 等价、全可行集双射或某个指定数值最优值。完整 certificate 的 529,636 obligations 与 448,596,660 work 已复核；此前缺少条件投影的成功请求仍保存在历史目录，不能冒充当前完整要求。来源为 `cases/test36/PROJECTION_REVIEW.md` 和 `audit/TEST36_COMPLETE_REVIEW.md`。

7.7 完整 corpus：联合 subflow 与 point contracts

`cases/test58/` 的完整 Source 有 2,500 个变量、1,750 行，Target 有 607 个变量、785 行。Forward 对所有 Source MI 点给出完整 Target MI 非增目标像，Reverse 对所有 Target MI 点给出完整 Source MI 等目标恢复。两方向保持最小值并提供恢复；不要求 LP 等价或全可行集双射。

其 V10 `SubflowLift` 同时产生 180 个节点、606 条有向弧的一个共同流，逐弧不超过给定原始流及总 demand 82，并保留所有所需节点 balance。它与 V4 FlowLift 的逐弧总流相等结论不同，也没有把独立局部流见证拼成未经检查的全局 tuple。所有 old-scope premises 先被核验，再进入同一 generated extension，最后核验完整 Target map 的全部 bounds、integer designations、rows 和 objective。联合存在性仍依赖 `docs/SUBFLOW_DRAFT.md` 的可信 builder 引理；完整 child obligations 不会独立证明这条引理。该实际完整请求复核为 8,448 obligations、2,029,879 work；八个保存的负例覆盖缺失 balance、越界 old-scope 引用和错误反向恢复。

point contract	实际核验内容	可推出的结论与边界
`Feasible`	选定实际模型的全部坐标、bounds、rows、所要求整数性和可选完整目标值	证明可行及该目标值；另有全局 lower-bound claim 才能证明最优值达到。
`Strictness`	先完成 full-coordinate permutation LP inclusion，再检查完整 output LP 点及其唯一对应 input 点	证明该 inclusion 的严格性；不能推广到部分投影。
`FiberStrictness`	完整 output LP 点，及固定全部 caller observations 后的完整 input LP fiber 反证	证明部分 coordinate/affine observation image 的严格性；不能用一个失败恢复点替代整个 fiber 的不存在性。
`ImageExclusion`，V10 草案	一个完整实际 LP 点、caller 绑定的完整另一模型 affine tuple，及其实际 bound/row 违反	证明这个完整像被排除；不需要全局 inclusion，也不建立全局 containment 或 strict inclusion。

`cases/test71/` 同时核验整数双向 identity/inverse、Target LP 到 Source LP 的完整 inclusion、严格性点，以及 Source integer 与 Target LP 的全局 lower bound -12 和达到该值的完整点。因此可报告这些精确最优值及严格 LP strengthening；整数等价并不意味着 LP 等价。完整 strictness 点每个坐标为 $1/56$，违反实际 Target 新行 $\sum x \le 12$。

`cases/test62/` 保留原有两方向整数目标保持和 V7 complete-domain bijection，再以 V10 `ImageExclusion` 核验 4,500-coordinate Source LP 点和全部 500 个 Target image expressions。检查到实际 Target 违反便成立；它不要求或证明 Source LP 的全局映射 inclusion。`cases/test18/`、`cases/test19/` 的完整 fiber refutations 则展示第三类 claim，定义和 fresh-scope 边界见 `docs/FIBER_POINT_V6.md`。

`cases/test80/` 展示 V9 的另一个边界：在完整原始 Source 上先证明全局 lower bound -160 并给

出达到值的完整 integer point。Source 到 Target 方向仅在 caller-bound Source objective level −160 上证明；Target 到 Source 方向在全部实际 Target integer 可行点上证明，并追加 image 属于 opposing Source level 的 equality。两个完整 map 的 affine inverse 同时被核验。得到的是所有 Source integer optimizers 与全部 Target integer 可行点的双射，不是所有 Source 可行点的双射；不是 V7 `bijections` counter 所统计的 paired-domain 机制。V9 与 V10 的这些实际成功请求均不冻结协议。

7.8 实际案例的可重复入口

上述五份完整实际 certificate 已使用当前重建 CLI 再次验证，命令和完整非压缩 reports 保存于 `docs/rpx/examples/corpus-verification.json`。这里的单次时钟结果仅作功能诊断，不作性能比较。已有产物可直接复核，例如：

Listing 7.15: 完整 test36 与 test58 的当前 verifier 入口

```
target/release/rpx verify cases/test36/source.mcm cases/test36/target.mcm cases/test36/proof.rpx
     cases/test36/request.json
target/release/rpx verify cases/test58/source.mcm cases/test58/target.mcm cases/test58/proof.rpx
     cases/test58/request.json
```

若要从 immutable corpus 文件重新导入、重组完整 RPA 并核对交付元数据，使用现有 delivery checker 的真实命令：

Listing 7.16: 完整模型与证书的交付复核

```
python3 tools/check_delivery.py --case test36 --case test58 --case test62 --kernel target/
    release/rpx --out docs/.build/manual-delivery.json
```

该工具与 preparation/receipt 都是复核编排。最终接受仍由完整实际模型、独立 request 和 RPX 进入 kernel 得出。当前 81/81 完整实际案例、11 个 family 与 83 次成功调用的交付结论来自 `audit/FINAL_DELIVERY_REPORT.md` 及 `audit/acceptance.json`；不同日期的冻结 checkpoint 和历史 scope 缺失记录保留其原始含义。

第八章　执行成本、资源预算与测量

本章描述当前 `crates/core/src/verify.rs`、`api.rs` 的计数与实现，以及仓库已保存的受控实验。它不把 D0 的 streaming tape、8-byte LiveFact、mmap 或 batch 设想当作现有性能。当前 CLI 读取 owned bytes，decoder 构建完整 owned model/certificate，再执行 verifier；只读输入、已解码对象、精确算术临时量和 proof scopes 均会占内存。

8.1　公开计数的确切含义

`VERIFIED` report 的 `statistics` 由 kernel 生成，不是作者提交的证书字段。下面列出主要计数；全部字段定义在 `Statistics`。

字段	当前计数边界
`obligations`	实际执行的 terminal/output checks，包括方向 Return、严格改进和 injectivity 等通过同一 obligation 入口的检查。Split 的两支均计数；不含 lift premises、local Cases Witness、point checks 或 fiber 内部检查。
`linear_terms`	Lin 展开所访问 premise expressions 的非零 coefficient 项数，不是 Lin 数或 scalar arithmetic 次数。
`substitution_terms`	substitute 展开每个被引用 map expression 时访问的非零项总数；不同 rows 的重复访问重复计数。
`branches`	ordinary Split 与 local Cases 的 branch points；一个二支 split 计一个 point，而非两个 leaf。
`peak_live_facts`	同一时刻当前 scope 可见的 logical local-register bindings 的峰值；不是累计定义数，也不是物理存储或 RSS。
`work`	被明确 charge 的操作总量，包括 step、expression/fact/map 项及常数、ancestor lookup probes、构造器和相关验证工作。是资源政策单位，不是 CPU instructions、运行时间或严格 bit-complexity 上界。
`points`, `point_coefficient_terms`	完整 point checks 及 point/observation 精确 evaluation 项数；不含 fiber 反证中的 Lin/代换项。Point 的 bounds/rows 检查不进入方向 Return 的 obligations。
`plotkin_pairs`	Plotkin witness 中已检查的 coordinate pairs；还存在 labels、membership 与 premise 扫描成本。
`flow_*`, `interval_*`, `menu_*`, `signed_*`, `subflow_*`	各类 lift 次数及其 generated variables、rows、terms。记录完整 generated extension，不代表存在性引理已被另一个 universal checker 证明。
`bijections`, `injectivity_*`	V7 paired-domain bijection 与其完整 injectivity domain/checks。Legacy `inverse` 的成功不自动增加此 bijections 计数。
`objective_level_*`, `halfspace_*`, `improvements`	条件域构造次数、生成行和项，以及严格改进 witnesses；V10 的新增 zero counters 会从 JSON 中省略。

Context 的 `charge` 做 checked addition，超过有效 work ceiling 就返回 `RESOURCE_LIMIT`。例如一次 static/local fact 读取还计入 expression 的 terms+1；local 引用另外计 ancestor 查找 probes。一次 map substitution 按引用表达式的 terms+1 charge。换一种内部计数政策可能改变 work，必须在实验中说明；同一 work 不能保证同一机器上耗时相同。

第七章的分段证书只有一组三项 Forward 数学要求，但 verifier 在两个 Return 各检查一次，所以 Forward 的 obligations 是六。完整 fiber refutation 使用同一 verifier 的 fresh-scope 调用，但 `point.rs` 只把其 work 加回共享预算，不把内部 branches、linear_terms、substitution_terms、peak_live_facts、plotkin_pairs 或 obligations 汇入顶层 Statistics；分析这类 proof 时不能从顶层零计数断言内部没有该工作。无限精度 rationals 的位长不同，也会使同样项数的 Lin 成本很不同。不能把任何一个单项计数当作完整成本。

8.2 共享预算与可复核的资源结果

当前默认 `Limits` 为 work 10^9、live facts 10^6、depth 64，root depth 为 1。一个 request 的全部方向、条件域、lift、injectivity、points/fiber refutations 和 improvements 共用 work；嵌套 scope 或新方向不会重置。CLI 可配置 work 与 live facts，depth 不提供 CLI override。

Listing 8.1: 预算是实际 verifier 参数

```
target/release/rpx verify --work 1000000000 --live-facts 1000000 cases/test58/source.mcm cases/
    test58/target.mcm cases/test58/proof.rpx cases/test58/request.json
```

有限预算是部署政策，不是模型数学定义。`INVALID`、`RESOURCE_LIMIT` 和 `IO_ERROR` 都不建立 reformulation 不成立。第七章已在同一正例上实际观察 `--work 0` 的 RESOURCE_LIMIT；test36 的同一完整产物在 10^9 work 政策下成功，旧 10^8 ceiling 下耗尽预算。这不是数学关系改变，也不能用失败运行时间作 speedup 的分母。

还必须分别记录其他层次的边界：CLI 每文件 owned read 为 512 MiB，RPA/request 文本为 256 MiB，serde JSON nesting capacity 为 128，binary nesting 为 64；模型 dimensions、counted arrays 和 rational magnitude 有各自上限。proof depth 与 JSON wrappers 的嵌套数不同。当前精确算术中间 numerator/denominator 的 bit budget 为 65,536。提高 work 不会绕过这些独立边界。

8.3 代换、精确累加与表示成本

对 output row r 和完整 map $y_j = E_j(x)$，直接代换展开

$$A_r E(x) - u_r = c_r + \sum_{j \in \mathrm{supp}(A_r)} a_{rj} E_j(x), \qquad W_r = \sum_{j \in \mathrm{supp}(A_r)} \mathrm{nnz}(E_j).$$

输入项可能重合、相消，输出可能变稠密；W_r 不只由 output row 的 nnz 决定。跨 row 重复的代换仍付出访问成本。当前 `Accumulator` 用 hash map 暂存精确 coefficient，再排序生成 canonical sparse expression；hash 只选择容器 bucket，精确 key/rational equality 决定接受。没有通用 affine-expression DAG、共享 mapping DAG 或证书中间结果缓存的格式承诺。

`BigRational` 保持精确数学语义。现有 denominator-one、乘 ±1 和零加值快路径避免部分通用算术工作，同时保留所需 intermediate/result checks；非 unit rational 仍使用 arbitrary-precision 运算。它不等于 D0 建议的机器小整数与大整数自动提升 ABI，也不把 coefficient 位长或 gcd 成本计成 CPU 指令。

展开长 prefix maps 可有二次物理大小；scalar injectivity proofs 也可能很大。二进制减少 JSON 文

本开销，并未保证完整 proof resident memory 与 peak live facts 成正比。一个解释性的成本分解是

$$T = T_{\text{read}} + T_{\text{decode}} + T_{\text{execute}} + T_{\text{process}}, \qquad T_{\text{execute}} = T_{\text{algebra}} + T_{\text{scopes}} + T_{\text{builders}} + T_{\text{checks}}.$$

它不是已证明的统一渐近上界；精确算术位长与保存的 physical representation 需要额外描述。

8.4 Shared scopes 的实现与内存边界

当前 `Facts` 借用不可变 parent，只在 child 保存 local definitions 与 deleted-ID masks。其可见集合为

$$L \cup (P \setminus D),$$

其中 local bindings 优先；drop/reuse/drop 不会复活祖先旧值。一个 child 的 DROP 不改变 sibling 或 parent。Cases 的两支独立检查，仅把共同结论插入 parent；ordinary Split 没有后续 continuation，left 检查后 right 可接管未改变的 parent 状态。这消除了每个 branch 复制全部 inherited facts 的成本。

祖先 lookup 的 probes 被计费，深度仍有界；local insertion 和 deletion-mask growth 使用 fallible reservation。`peak_live_facts` 只计当前可见 bindings。被 child mask 隐藏的 immutable ancestor payload 仍在内存中，masks 也占空间，所以 logical live limit 不能直接转换成固定 bytes/live-fact 或 peak RSS 上限。decoder 的 owned objects、arithmetic allocations 及部分依赖库 allocation failure 也是当前内存保证的边界。

独立 copied-BTreeMap state oracle 的 127,072 次操作、depth-64 checks 与 negative scope tests 见 `audit/SHARED_SCOPE_REVIEW.md`；生产集成另见 `audit/scope_v6_merge.json`。状态正确性审阅和 latency 测量分别保存，不能互相代替。

8.5 时钟字段与端到端成本

当前 Rust façade 和 CLI 使用同一报告函数，公开 microseconds 字段：

字段	含义
`read_us`	读取 source、target、RPX 和 request；filesystem-free `verify_bytes` 不返回此字段。
`model_decode_us`	两份完整 MCM 的 decoding/validation 阶段。
`proof_decode_us`	RPX decoding 加 caller request text parsing。
`execute_us`	完整 `verify_request_with_diagnostics` 调用，包含全部请求验证。
`total_us`	kernel API 本次调用的内部总时间，包括报告前的操作；不含 process startup/shutdown。
process wall time	外部计时，另含进程启动和退出。不是另一个 kernel 公开计数字段。

MPS/LP import、作者生成 RPA、assembler 和任何外部 oracle 必须单列。`CANONICALIZED`/`ASSEMBLED` 的 `total_us` 不能当作 verification time。未重测的历史 preparation 应标为 archived/unmeasured；raw 数据中的空 preparation array 或 sentinel 0 不能解释为零成本。

可靠的对比先固定完整数学 request、模型/proof bytes、source/binary/lockfile identities、硬件、编译选项和 budget，再记录 cold/warm 文件状态、重复次数、顺序与 isolation。每个样本都核对退出码和 VERIFIED，保存失败和 outliers；timing 与 RSS 另起 processes。至少报告 min/median/max 或完整分布，并同时保存计数、模型 dimensions、artifact size 与阶段时间。

8.6 已有实验：明确保留适用范围

8.6.1 共享状态与精确算术

bench/SHARED_SCOPE_REPORT.md 固定了 frozen V5 的 before/after sources，数学规则和 certificate bytes 相同，仅改变 scope 表示及相应 work accounting。三个 copying-heavy workloads 的 execution medians 如下，单位 ms：

active facts / Cases	before	after	work before	work after
256 / 256	3.439	0.425	267,535	8,720
1,024 / 1,024	51.413	1.605	4,215,823	34,832
4,096 / 4,096	827.142	6.517	67,194,895	139,280

这支持消除所选二次复制成本，不支持普遍 127 倍加速。完整 broader matrix 保存了 nonbranching regressions：例如 real65 execution median 从 3690.334 增到 3793.140 ms；logical counters 中除了 work 以外的数学项、全部 physical artifacts 一致。test18 后来的成功是原字节的预算内 replay，其原失败没有成功性能分母。

bench/RATIONAL_SHORTCUTS_REPORT.md 对 49 个 paired workloads、1,176 个成功 verifier processes 观察同 bytes、同完整计数下的算术快路径。所有 execution medians 降低，但 startup 主导的 process latency 不全部降低；例如 injectivity-64 从 2.186 增到 2.391 ms。RSS median 变化在 3% 内。报告描述其固定 prototype binary，不能把后来 resource/API 修订后的可执行文件自动等同于被测候选。生产合并边界与额外检查记录在 audit/V7_ARITHMETIC_MERGE.md。

8.6.2 V10 旧任务 regression 与顺序敏感性

bench/V10_REGRESSION_REPORT.md 保留 31 个 paired 旧任务、712 个成功 verification/RSS processes，相同 artifacts、request 和完整 non-timing results。candidate 在 19 个任务的 execution median、23 个任务的 full-process median 上增加。test65 execution 为 1792.479 到 1864.392 ms，约 +4.0%；rational-1000 为 0.792 到 0.972 ms，增加 0.180 ms。没有通用加速或无 regression 声明。

预先声明的一次 alternating-order follow-up 保存另外 72 个 timed calls 和八个 excluded warmups，四个任务的 paired median difference 分别为 rational-1000 +0.003 ms、test65 +82.552 ms、test71 −0.038 ms、test79 −0.005 ms。test65 的正成本保留为交付限制，其他三项原有较大差异未在本轮持续。分布和 within-pair order effects 不支持把一个精确百分比归因于代码。原矩阵未被替换，也没有新的 RSS 声明；原始报告见 bench/results/v10-order-sensitivity/ORDER_SENSITIVITY_REPORT.md。

8.6.3 新能力绝对 baseline

bench/NEXT_CAPABILITY_DRAFT_BASELINE.md 固定 22 个任务，264 个成功 verification/RSS processes，并单列 63 次 synthetic author/assembly preparations。使用最终 cold checkpoint 的同一 binary，task-owned build/preparation/acceptance 工作在采样期间 idle；warm files、fresh process，OS activity 未控制。新任务不与无法证明相同命题的旧 binary 计算 speed ratio。

两个已有报告的 median 样本如下；完整 min/median/max、decoder 时间和身份都在原报告：

任务	execute ms	process ms	peak RSS MiB
完整 actual test58	18.369	28.749	31.891
subflow 1,024 nodes / 4 groups	56.732	87.707	69.141

第二项的完整 Python preparation median 为 9.154 s，包含简单 $O(nm)$ author construction；不能把这个成本藏在 constant-time theorem 的叙述中。actual test58 的历史 author preparation 未重测，表中没有零成本含义。Corpus 的总 functional replay 耗时和本手册单次示例 replay 都不是该受控矩阵的替代。

8.7 复现与下一次优化的登记

仓库已有 fixed matrix drivers；新的运行须用未使用的 label 保存 raw evidence。下面的命令可建立当前 binary 的新绝对测量，不会复写历史样本：

Listing 8.2: 当前完整初始与 extended matrices

```
python3 tools/benchmark.py --kernel target/release/rpx --source-root . --label manual-current-
    initial
python3 tools/benchmark_extended.py --kernel target/release/rpx --source-root . --label manual-
    current-extended
```

正式执行前应安排独立采样窗口并登记机器与其它任务状态；这些命令不自动保证 isolation。固定 V10 final-driver 与能力 config 还验证专门 binary/manifest hashes，因此重建当前 binary 不会被默认为同一历史样本。已有 driver 拒绝覆盖 raw evidence，应换新 label 或单独建立新固定计划。

共享 DAG、dedicated batch、streaming decoding、模型缓存和独立 solution reconstruction 是可能的后续工作，均非当前报告中的既有收益。新增 batch 必须另定义可观察语义、operand 检查、scope/obligation 与失败条件，不能用名称加入隐藏 theorem。优化接受路径继续做精确验证；浮点 residual、probabilistic hash 和外部 solver success 不构成接受条件。

任何对其它 proof format 的速度比较，都须提供相同数学请求和推导、相同线程政策及 preparation/verification 边界。当前报告没有这种受控 cross-format 实验，因而不宣称相对 VIPR 或求解器的性能优势。

历史 source snapshots 与部分 prototype 完整副本已在交付后的清理中删除，原 manifests、receipts、logs 与 diffs 保留；具体清理边界见 `audit/POST_DELIVERY_CHANGES.md`。依赖这些副本的历史重建必须先恢复原版本数据。本章的新测量入口使用当前源码，不能把重新编译的当前 binary 当作原实验 binary。

附录 A　设计沿革、冻结范围与后续登记

本手册以当前已实现 kernel 为对象。原 D0 是 2026 年 10 月 9 日整理的设计输入，其 `.rpa 0.1`、28-opcode tape、128-byte 候选 header、digest 绑定、FactId SSA 和分支汇合不是已实现格式。MCM 1、RPA/RPX 1–8 的冻结含义与 V9/V10 的工作草案状态必须分别登记；不能因为 corpus 已完成便追认 D0 ABI 或宣布所有协议冻结。

A.1　D0 提案 P-01 至 P-07 的实际处置

保留原提案编号以便追溯。以下为当前实现与原建议的对应关系，既非待表决草案，也不表示原字段编码已经发布。

编号	处置	已实现行为与差异
P-01	数学原则落地，接口重写	目标统一为完整 affine minimization，包括常数。caller request 独立且必须与证书完全相等；MI/LP 用独立 input/output integer booleans 表达，不是 D0 两个 profile。双向 Equal 保留 attainable objective values 与 extended infimum；双向 NonIncreasing 保持 extended infimum 和非空性。映射无需互逆；bijection/projection 是另行绑定并核验的要求。
P-02	用完整 terminal maps 替代	每支从同一 parent 的 isolated fact view 开始。每个 Return 一次给出全部 output coordinates，核验全部 obligations；Split 是 terminal，没有 post-merge continuation。无 DONE 交集、DEFER、三态 map 或 SEALED_PIECEWISE。V2 Cases 仅导出共同的已核验 proposition，不能导出独立 existential output fragments。
P-03	不采用 D0 SSA 编号	Local 是 mutable u32 register。定义不能覆盖 live binding；DROP 后同一 ID 可重新定义，引用立即解析。没有按语法深度优先预分配的历史 FactId slots。child 的 local writes/deletions 隔离，borrowed ancestors 与 deletion masks 保留 state invariant。
P-04	数学闭包落地于 Lin	`Lin` 的 Int kind 允许有理数倍 EQ premises；constant 和 INT multipliers 必须整数。EQ=0 的项不改变整数性。启用 input integrality 才能读取 declared integer predicates。由模型 facts 的推导与 Exact 的结构整数性判定不同。
P-05	精确简写落地，编码不同	`Exact` 是 terminal/derivation witness Ref，kernel 自行检查 constant-zero EQ、constant-nonpositive LE 或有整数 domains 支持的 integral affine expression。不是 CHECK flag，不产生 local fact，也不能作为 Lin premise。
P-06	覆盖原则落地，重建接口未发布	Real Split/Cases 的 closed guards 为 $e \le 0$ 与 $-e \le 0$；integer guards 为 $e \le 0$ 与 $1 - e \le 0$，须有匹配 INT。两个完整子树都被 decoder/verifier 检查。D0 的独立数值解应用与零点 le 优先 CLI 未实现；其建议没有成为接受规则。

编号	处置	已实现行为与差异
P-07	候选 wire layout 被替代	当前采用 16-byte common header、little-endian length/count 与嵌套 records，canonical rationals 和 complete model/claim objects。无 ScalarRef immediate、pool/directory tape 或可选 metadata section。实际 tags、version gates、reserved bytes、preflights 见第四章；原候选字节向量不能作当前 golden vector。

这些处置保留目标：独立模型绑定、精确有限证书、完整双向语义、可审阅失败与资源边界。实际模型绑定依赖每个 input premise 与完整 output obligations 的语义核验，RPX 不含允许跳过模型检查的摘要。准备脚本和 manifest hash 是来源证明，不是数学公理。

A.2 实现版本 V1 至 V10 的能力历史

各 increment 的数学与 byte meanings 沿原文保留。表中的版本是使用相应 feature 的最低 RPX 版本；assembler 通常输出实际必要的最低版本，RPA N 标头仅允许 N 范围的 features。不是每个版本单独运行一个 verifier。

版本	协议状态	新增能力与边界
V1	已冻结	完整模型 affine directions；Lin/Round/Drop，完整 Return、exhaustive Split、Absurd；独立 caller claims，objective/projection/legacy affine inverse。MCM 1 精确模型与严格 JSON RPA 1。来源：`SEMANTICS_V1.md`、`FORMAT_V1.md`、`RPA_V1.md`。
V2	已冻结	EqPair 的精确 antisymmetry，local Cases 的共同 proposition derivations；保留全部 simultaneous output obligations，避免重复无关整模型分支。实际 test88 integer alias 和 test84 parity 见 `FORMAT_V2.md`。
V3	已冻结	静态 Plotkin theorem、caller objective lower bound、完整 Feasible 和 full-permutation Strictness points。test71 核验 integer 等价、严格 LP strengthening 和精确 source-integer/target-LP 最优值。静态 theorem 无 host callback。来源：`FORMAT_V3.md`、`POINT_V3.md`。
V4	已冻结	Common-source joint FlowLift：所有 commodities 一起满足每个节点 balance，并保留每条弧总流及 circulation。test28 不把单独局部存在性当作 joint tuple。来源：`FORMAT_V4.md`、`FLOW_V4.md`。
V5	已冻结	Independent IntervalLift 与 caller affine observations；完整 old-scope premise checks 后联合生成所需连续坐标。test23 使用 signed affine observations。来源：`FORMAT_V5.md`、`INTERVAL_V5.md`。
V6	已冻结	FiberStrictness 和完整 point refutation payload；固定全部 caller coordinate/affine observations，在实际完整 input LP fresh scope 中反证整个 fiber。Shared-scope production integration 是表示/资源优化，非新数学 theorem。test18/test19 见 `FORMAT_V6.md`、`FIBER_POINT_V6.md`。
V7	已冻结	Optional MenuLift 和 explicit complete-domain bijection pairs；forward-after-reverse identities 与 paired-domain injectivity 覆盖全部坐标。test59/test60 核验整数 menu 双射与 LP observation images。来源：`FORMAT_V7.md`、`MENU_V7.md`。

版本	协议状态	新增能力与边界
V8	已冻结	Activated signed real decomposition，同时生成 p, n, u, v，证明 $p-n=F$、$u+v=Z$ 和容量 bounds。必须 integer activation，而 real flow 可以连续。来源：`FORMAT_V8.md`。
V9	已实现、未冻结	Caller-bound `input_objective_level` 把完整 input domain 与完整 objective equality 相交；legacy opposing inverse 要额外 image-level membership。条件 bound 不能充当 global bound；受限 claim 不偷换旧 whole-domain point/bijection 的含义。test80 与 `FORMAT_V9_DRAFT.md`。
V10	已实现、未冻结	严格 positive-domain improvement、完整 affine ImageExclusion、joint demand-bounded SubflowLift，以及 caller-bound original-coordinate `input_halfspace`。追加 records 与 feature flags 独立 gating；兼容已有有效字节。实际 test36/test58/test62/test83，来源：`FORMAT_V10_DRAFT.md`、`SUBFLOW_DRAFT.md`。

V1–V8 的 checkpoint test/case counts 是历史时点的证据，不应改写成今天的总量。例：frozen V8 rebuild 是 198 tests、当时列举的 34 个实际 certificate；后来的交付 cold build 是 262 tests。两个记录各有 source/binary/lock manifest，不能用后者倒填前者。`docs/CURRENT_FORMAT.md` 汇总 V1–V8 并单列 V9，V10 完整草案补上其新增规则；这些文档的合并不创造新版本，也不冻结 V9/V10。

当前 façade 的 `api.rs` 与 CLI 共用 report/status/exit handling，提供文件及 filesystem-free verification、canonicalization、assembly、inspection 和 model export。这是交付 snapshot 后的 interface 整理，未改 wire formats 或数学规则。精确差异在 `audit/POST_DELIVERY_CHANGES.md`；当前机器可读 `code/location` 为诊断，不给 proof 更多接受权。

A.3 D0 开放项 O-01 至 O-05 的当前状态

编号	当前状态	已解决部分与保留边界
O-01	MCM 1 已实现并冻结	独立 canonical model 文件、rational/name/dimension validation 与 encoder/decoder 已公开。原 SHA 覆盖范围问题不再是接受前提，因为 proof 采用全模型语义绑定。`export-model` 输出加载后的全部 canonical JSON，不是 alternate acceptance representation。
O-02	已定义可信输入子集	`MPS_V1.md` 与 `LP_V1.md` 明确 defaults、bounds/ranges、names、exact decimal、objective sense 和 unsupported constructs。全部 159 个 corpus MPS 与三个 LP 已导入；LP 与独立 Fraction parser 比对全部字段。支持这个子集不承诺所有厂商 dialect、SOS、indicator、quadratic 或 semicontinuous semantics。
O-03	独立数值解工具未发布	Return 与 piecewise proof 的数学 total-map 语义已核验，完整 inverse/bijection 的要求已实现。当前 CLI 没有统一 numeric solution application/export 命令；不能把 verify、disassemble 或 model export 当作该工具。新的应用接口需另定义 guards、扩展 witness 实例化与 exact output。

编号	当前状态	已解决部分与保留边界
O-04	有限静态扩展已实现，通用 batch 未实现	V3–V10 通过 reviewed static constructors/checkers 扩展能力，未知 tags 与不支持的版本拒绝。无 theorem registry、动态插件、case whitelist 或 host callback。通用 expression/map DAG、dedicated batches、capability negotiation 仍待独立设计和评审。
O-05	有明确当前政策与真实 benchmarks	当前预算、CLI overrides、四类 verifier outcomes、固定矩阵和完整 raw evidence 已落地。默认 work 从旧 10^8 提到 10^9 是有记录的 resource-policy 变更，不是速度优化。严格 bit-complexity/peak-RSS 上界、全部 allocation failure 可恢复性和所有未来任务均能预算内验证没有被证明。

O-01/O-02 已不阻止当前工具链互通；O-03/O-04 的未来 representation/application 工具也不是当前 81 个实际任务完整验收的前提。资源政策与数学 semantics 仍需区分：预算耗尽不使完整证书成为逻辑反证。

A.4 V9/V10 冻结前继续保留的边界

已完成的真实模型验收和 byte/limit evidence 支持工作实现，协议发布仍需要独立明确的冻结决定。冻结登记至少核对：

1. V9 objective-level 与 V10 halfspace 原始坐标域、相交次序、生成行顺序及 opposing legacy inverse membership；受限 claim 与旧 point/bijection 的禁止组合。
2. PositiveRefutation 的非严格 $P \le 0$ 结论与旧 scope $P \ge 0$ assumption、ImproveReturn 的正 margin，以及所有 positive-domain points 的完整 simultaneous self-model map。
3. ImageExclusion 的完整 caller-bound map、完整实际 LP 点、actual bound/row violation；不得宣传为未检查的 global containment。
4. Subflow 的 demand-bounded joint-existence 引理、每一组所有 old-scope premises、完整 generated schema 和 output tuple；builder 仍是可信 inference code。
5. 每个新增 record 的版本 gating、flags/reserved bytes、count/remaining-payload preflights、depth/work/live limits；所有 syntactic branches 均受检查，包括可能矛盾的 branches。
6. 既有 V1–V8 和早期 V9/V10 有效字节含义与 absent-field bytes 保持；历史 78 requests 的兼容 receipts 保留，不能以重建文本替代原字节对照。
7. 完整 actual-case caller scope：成功提交的弱 request 与 original-report 完整要求必须分开复核，不能以模型/proof hashes 或 author scope hints 掩盖遗漏的 property。
8. Fresh locked source build、完整 tests、immutable checkpoint、cold actual-model replay、独立 scope review 和受控成本矩阵分别有身份记录；功能成功不等于 protocol freeze 或普遍性能承诺。

最终交付已经完成 81/81 完整实际 cases、11 个 families、83 次成功 kernel calls；test46 三个独立 direction files 的 conjunction 在 delivery metadata 中明确登记，没有 cross-file facts/lemmas。原 78 个 submitted requests 也维持 byte/non-timing compatibility。这些已完成事项见 `audit/FINAL_DELIVERY_REPORT.md`、`audit/acceptance.json`、`audit/contract_scope_gates.json`。旧 `DELIVERY_ACCEPTANCE.md` 的 76/81 与个别历史 README 的 pending 文句记录此前 scope 审查，不能取代最新 final ledger。

可信边界包含输入与证书解析、精确算术库、表达式与证明检查器、静态扩展 builder，以及编译器、运行时和操作系统。独立 oracle、数学论证、攻击测试与完整语料互为补充，不构成 Rust 实现的形式化证明。

A.5 手册修订记录与维护准则

日期	修订	内容
2026-10-09	D0 provenance	整理原 28-opcode v0.1 候选设计、P-01–P-07、候选 wire layout 与 placeholder-hash 示例。它是设计来源，没有成为已实现 ABI。
2026-10-13	当前实现手册	按 safe Rust kernel 重写模型/请求、scalar/local/theorem rules、joint maps、MCM/RPX records、严格 JSON RPA、CLI/API、预算与诊断；完整区分 frozen V1–V8 和 working V9/V10。
2026-10-13	可执行证据	新增三份完整双向 LP/RPA/request，实跑三正例及回读、九次 INVALID 逻辑变异、三次 RESOURCE_LIMIT；另复核完整 test36/test58/test62/test71/test80，保存实际产物与报告身份。

维护本手册时，先检查当前 code 的 public types、decoder/version gates 和实际 checker，再同步 request grammar、全部义务、例子与失败分类。性能优化要保存固定任务和原始数据，数学扩展要给明确规则与可信边界，冻结状态要有独立决定。更新中文手册不回写历史 checkpoint 的证据，也不从完整 corpus 的成功推导所有可能 MILP 改写都有紧凑可接受证书。